Student Dropout Risk Scoring with Regression for Early Intervention

Objective

The objective of this analysis is to extend the previous `student dropout prediction` experiment by reframing the problem as a continuous risk estimation task for early intervention.

Instead of producing only a binary classification outcome, the selected model is used to generate a continuous dropout-related risk score between 0 and 1 for each student. This score is later expressed as a percentage to support easier interpretation by non-technical stakeholders.

The analysis follows a structured workflow consisting of:

1. Data Preparation
2. Risk Model Training
3. Regression-Oriented Score Evaluation
4. Continuous Risk Score Construction
5. Risk Distribution Analysis
6. Risk Decile Analysis
7. Risk Category Design
8. Calibration Analysis
9. Support Capacity Simulation
10. Practical and Ethical Interpretation

This notebook uses the same binary observed outcome as the previous analysis, but reframes the modeling task as continuous risk estimation. A regression model is trained on the binary target coded as 0 for Graduate and 1 for Dropout, allowing the output to be interpreted as a continuous dropout-related risk score between 0 and 1.

Although a regression model is used in this notebook, the observed target remains binary. For this reason, the task should not be understood as traditional regression with a naturally continuous outcome. Instead, the regressor is used as a score-generating method that produces a continuous dropout-related risk estimate on a 0 to 1 scale.

1. Import Libraries

In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor

from sklearn.metrics import (
  mean_absolute_error,
  mean_squared_error,
  r2_score,
  brier_score_loss,
)

from sklearn.calibration import calibration_curve

sns.set(style="whitegrid")
```

2. Load Dataset

In this notebook, the same dataset is used as in the previous student dropout analysis. However, the goal is now different. Instead of focusing only on binary classification, the dataset is used to build a `regression-based risk scoring framework` that estimates a continuous dropout-related risk score between 0 and 1.

2.1. Load Dataset

In [2]:
```
df = pd.read_csv("../datasets/data.csv", sep=";")
df.head()
```

Out[2]:

	Marital status	Application mode	Application order	Course	Daytime/evening attendance\t	Previous qualification	Previo qualificati (grac
0	1	17	5	171	1	1	12
1	1	15	1	9254	1	1	16
2	1	1	5	9070	1	1	12
3	1	17	2	9773	1	1	12
4	2	39	1	8014	0	1	10

5 rows × 37 columns

2.2. Clean Column Names

Before continuing with the analysis, the column names are cleaned to remove unnecessary whitespace, tab characters, and quotation marks.

In [3]:
```
# Clean column names
df.columns = df.columns.str.replace("\t", "")
df.columns = df.columns.str.strip()
```

```
df.columns = df.columns.str.replace("'", "")

df.head()
```

Out[3]:

	Marital status	Application mode	Application order	Course	Daytime/evening attendance	Previous qualification	Previo qualificati (grad
0	1	17	5	171	1	1	12
1	1	15	1	9254	1	1	16
2	1	1	5	9070	1	1	12
3	1	17	2	9773	1	1	12
4	2	39	1	8014	0	1	10

5 rows × 37 columns

3. Dataset Overview

Before preparing the data for risk scoring, it is important to understand the overall structure of the dataset. This includes checking the number of observations and features, inspecting the variable names and data types, verifying whether missing values are present, and reviewing the distribution of the target variable.

3.1. Dataset Shape

In [4]:
```
df.shape
```

Out[4]: (4424, 37)

3.2. Column Names

In [5]:
```
df.columns
```

Out[5]:
```
Index(['Marital status', 'Application mode', 'Application order', 'Course',
       'Daytime/evening attendance', 'Previous qualification',
       'Previous qualification (grade)', 'Nacionality',
       'Mothers qualification', 'Fathers qualification', 'Mothers occupation',
       'Fathers occupation', 'Admission grade', 'Displaced',
       'Educational special needs', 'Debtor', 'Tuition fees up to date',
       'Gender', 'Scholarship holder', 'Age at enrollment', 'International',
       'Curricular units 1st sem (credited)',
       'Curricular units 1st sem (enrolled)',
       'Curricular units 1st sem (evaluations)',
       'Curricular units 1st sem (approved)',
       'Curricular units 1st sem (grade)',
       'Curricular units 1st sem (without evaluations)',
       'Curricular units 2nd sem (credited)',
       'Curricular units 2nd sem (enrolled)',
       'Curricular units 2nd sem (evaluations)',
       'Curricular units 2nd sem (approved)',
       'Curricular units 2nd sem (grade)',
       'Curricular units 2nd sem (without evaluations)', 'Unemployment rate',
       'Inflation rate', 'GDP', 'Target'],
      dtype='str')
```

3.3. Data Types

In [6]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4424 entries, 0 to 4423
Data columns (total 37 columns):
 #   Column                                          Non-Null Count  Dtype
---  ------                                          --------------  -----
 0   Marital status                                  4424 non-null   int64
 1   Application mode                                4424 non-null   int64
 2   Application order                               4424 non-null   int64
 3   Course                                          4424 non-null   int64
 4   Daytime/evening attendance                      4424 non-null   int64
 5   Previous qualification                          4424 non-null   int64
 6   Previous qualification (grade)                  4424 non-null   float64
 7   Nacionality                                     4424 non-null   int64
 8   Mothers qualification                           4424 non-null   int64
 9   Fathers qualification                           4424 non-null   int64
 10  Mothers occupation                              4424 non-null   int64
 11  Fathers occupation                              4424 non-null   int64
 12  Admission grade                                 4424 non-null   float64
 13  Displaced                                       4424 non-null   int64
 14  Educational special needs                       4424 non-null   int64
 15  Debtor                                          4424 non-null   int64
 16  Tuition fees up to date                         4424 non-null   int64
 17  Gender                                          4424 non-null   int64
 18  Scholarship holder                              4424 non-null   int64
 19  Age at enrollment                               4424 non-null   int64
 20  International                                   4424 non-null   int64
 21  Curricular units 1st sem (credited)             4424 non-null   int64
 22  Curricular units 1st sem (enrolled)             4424 non-null   int64
 23  Curricular units 1st sem (evaluations)          4424 non-null   int64
 24  Curricular units 1st sem (approved)             4424 non-null   int64
 25  Curricular units 1st sem (grade)                4424 non-null   float64
 26  Curricular units 1st sem (without evaluations)  4424 non-null   int64
 27  Curricular units 2nd sem (credited)             4424 non-null   int64
 28  Curricular units 2nd sem (enrolled)             4424 non-null   int64
 29  Curricular units 2nd sem (evaluations)          4424 non-null   int64
 30  Curricular units 2nd sem (approved)             4424 non-null   int64
 31  Curricular units 2nd sem (grade)                4424 non-null   float64
 32  Curricular units 2nd sem (without evaluations)  4424 non-null   int64
 33  Unemployment rate                               4424 non-null   float64
 34  Inflation rate                                  4424 non-null   float64
 35  GDP                                             4424 non-null   float64
 36  Target                                          4424 non-null   str
dtypes: float64(7), int64(29), str(1)
memory usage: 1.2 MB
```

3.4. Missing Values Check

In [7]:
```
missing_values = df.isnull().sum()
missing_values[missing_values > 0]
```

Out[7]:
```
Series([], dtype: int64)
```

The dataset was checked for missing values before further preprocessing. The results show that no features contain missing data. Therefore, no imputation is required for this analysis.

3.5. Target Variable Distribution

The original dataset contains **three** possible academic outcomes:

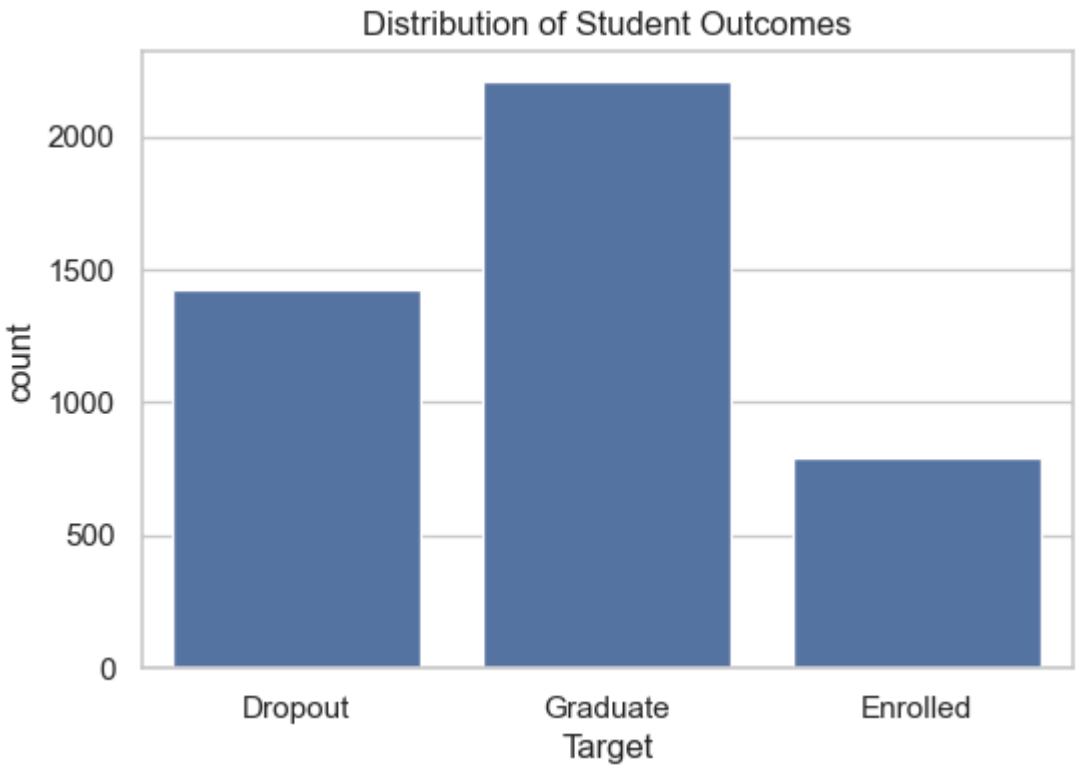
- Dropout
- Enrolled
- Graduate

Since the purpose of the project is to study dropout-related risk using known final outcomes, the distribution of the original target variable is first inspected before filtering is applied.

In [8]:
```
df["Target"].value_counts()
```

Out[8]:
```
Target
Graduate    2209
Dropout     1421
Enrolled     794
Name: count, dtype: int64
```

In [9]:
```
plt.figure(figsize=(6,4))
sns.countplot(x="Target", data=df)

plt.title("Distribution of Student Outcomes")
plt.show()
```



3.6. Interpretation

The dataset includes three academic outcomes, but only two of them represent known final states. Since the objective of this notebook is to estimate continuous dropout-related risk from known historical outcomes, the next section prepares the data by filtering the target variable, removing leakage-related features, encoding categorical variables, and constructing the ethical feature set used for regression-based risk scoring.

4. Data Preparation

Before generating risk scores, the dataset must be prepared in a way that remains consistent with the previous notebook. This includes filtering the target variable, removing features that may introduce data leakage, reducing unnecessary categorical complexity, encoding categorical variables, and excluding sensitive demographic and socioeconomic attributes.

4.1. Filter Target Variable

The `Enrolled` category is removed because it does not represent a known final academic outcome. The remaining target is coded as 0 for Graduate and 1 for Dropout.

Although the observed target remains binary, it is used here as the basis for a regression-based risk estimation task. Because the target is coded on a 0 to 1 scale, the regression model can later produce a continuous output that is interpreted as dropout-related risk.

In [10]:
```
# Remove whitespace from target values
df["Target"] = df["Target"].str.strip()

# Remove records with unknown final outcome
df = df[df["Target"] != "Enrolled"]

# Convert target to binary
df["Target"] = df["Target"].map({
  "Dropout": 1,
  "Graduate": 0
})

df["Target"].value_counts()
```

Out[10]:
```
Target
0    2209
1    1421
Name: count, dtype: int64
```

In [11]:
```
print(df["Target"].unique())
```

```
[1 0]
```

After removing the `Enrolled` category, the target variable becomes a binary observed outcome that can be used as the basis for regression-based risk estimation.

4.2. Remove Second Semester Features

Because the goal of the project is **early risk detection**, only variables that would realistically be available at an early stage of the student journey should be used.

`Second-semester` features may introduce data leakage, since they contain information from a later point in time and could make the model unrealistically strong. For this reason, they are removed before training the risk model.

In [12]:
```
df = df.drop(columns=[
  "Curricular units 2nd sem (credited)",
  "Curricular units 2nd sem (enrolled)",
  "Curricular units 2nd sem (evaluations)",
  "Curricular units 2nd sem (approved)",
  "Curricular units 2nd sem (grade)",
  "Curricular units 2nd sem (without evaluations)"
])

df.shape
```

Out[12]:
```
(3630, 31)
```

4.3. Define Feature Matrix and Target Variable

After filtering the target variable and removing leakage-related features, the dataset is divided into:

- X = predictor variables
- y = target variable

In [13]:
```
X = df.drop("Target", axis=1)
y = df["Target"]
```

4.4. Reduce High-Cardinality Categories

Some categorical variables contain many unique categories. To reduce `dimensionality`, rare categories are grouped into an **Other** category while the most frequent categories are retained.

This helps simplify the feature space without removing the variables entirely.

In [14]:
```
def group_rare_categories(df, column, top_n=10):
  top_categories = df[column].value_counts().nlargest(top_n).index
  df[column] = df[column].apply(lambda x: x if x in top_categories else "Other")
  return df
```

In [15]:
```
high_cardinality_cols = [
  "Course",
  "Nacionality",
  "Mothers occupation",
  "Fathers occupation",
  "Mothers qualification",
  "Fathers qualification"
]

for col in high_cardinality_cols:
  df = group_rare_categories(df, col, top_n=10)

print("Unique values after grouping:")
for col in high_cardinality_cols:
  print(col, ":", df[col].nunique())
```

```
Unique values after grouping:
Course : 11
Nacionality : 11
Mothers occupation : 11
Fathers occupation : 11
Mothers qualification : 11
Fathers qualification : 11
```

4.5. Apply One-Hot Encoding

Some variables in the dataset are categorical but encoded as numeric category labels. These should not be treated as continuous numerical values, because they do not represent meaningful ordered relationships.

`One-hot encoding` is therefore applied to convert these categorical variables into binary features.

In [16]:
```
categorical_cols = [
  "Marital status",
  "Application mode",
  "Course",
  "Nacionality",
  "Mothers qualification",
  "Fathers qualification",
  "Mothers occupation",
  "Fathers occupation"
]
```

In [17]:
```
X = df.drop("Target", axis=1)
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

print("New shape:", X.shape)
```

```
New shape: (3630, 104)
```

4.6. Apply Ethical Feature Selection

In the previous notebook, an ethical comparison was performed by removing potentially sensitive demographic and socioeconomic attributes. Since this notebook focuses on

responsible and interpretable risk scoring, the ethical version of the feature set is used here.

The following sensitive attributes are excluded:

- Gender
- Nacionality
- Mothers qualification
- Fathers qualification
- Mothers occupation
- Fathers occupation

This helps reduce the risk that the model relies directly on demographic identity or family background when generating support-oriented risk scores.

In [18]:
```
sensitive_prefixes = [
  "Gender",
  "Nacionality",
  "Mothers qualification",
  "Fathers qualification",
  "Mothers occupation",
  "Fathers occupation"
]

cols_to_drop = [
  col for col in X.columns
  if any(col.startswith(prefix) for prefix in sensitive_prefixes)
]

X_eth = X.drop(columns=cols_to_drop)

print("Original number of features:", X.shape[1])
print("Ethical dataset number of features:", X_eth.shape[1])
```

```
Original number of features: 104
Ethical dataset number of features: 53
```

4.7. Train/Test Split

To evaluate the risk model on unseen data, the ethical dataset is split into training and testing sets. Stratification is used so that both sets preserve the original class distribution.

In [19]:
```
X_train, X_test, y_train, y_test = train_test_split(
  X_eth,
  y,
  test_size=0.2,
  random_state=42,
  stratify=y
)

print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

```
Training set: (2904, 53)
Test set: (726, 53)
```

4.8. Interpretation

After preprocessing, the dataset is ready for risk model training. Although the observed target remains a binary dropout outcome, the purpose of this notebook is no longer to produce only a hard class label. Instead, the selected regression model will generate a continuous dropout-related risk score between 0 and 1.

This score can later be converted into a percentage, used to rank students by estimated risk, and grouped into practical categories for support-oriented interpretation.

5. Train the Ethical Gradient Boosting Regressor

In the previous notebook, several original and ethical models were trained and compared. In this notebook, the goal is no longer to compare multiple classification algorithms, but to train a single model that can generate an interpretable continuous dropout-related risk score.

For this purpose, the ethical gradient boosting approach is used in regression form. The model is trained on the target coded as 0 for Graduate and 1 for Dropout, allowing it to produce a continuous output between 0 and 1 that can later be interpreted as dropout-related risk.

5.1. Model Selection Justification

The `Ethical Gradient Boosting` approach is selected for three main reasons:

- it performed strongly in earlier experiments
- it captures non-linear relationships in structured tabular data
- it can be used here in regression form to generate a continuous dropout-related risk score rather than only a hard class label

This makes the model especially suitable for early intervention analysis, where the goal is not only to classify students into outcome groups, but also to estimate how strongly each student profile is associated with historical dropout patterns.

5.2. Train the Model

The selected `Gradient Boosting Regressor` model is trained using the **ethical** training dataset prepared in the previous section.

In [20]:
```
# Initialize the selected risk model
risk_model = GradientBoostingRegressor(
  n_estimators=200,
  learning_rate=0.05,
  random_state=42
```

```
)

# Train the model
risk_model.fit(X_train, y_train)
```

Out[20]:

```
▾ GradientBoostingRegressor
▸ Parameters
```

5.3. Make Predictions

After training the model, continuous risk scores are generated for the test set.

Because the regressor is trained on a target coded as 0 for Graduate and 1 for Dropout, each predicted value can be interpreted as a dropout-related risk score. Values closer to 1 indicate stronger similarity to historical dropout cases, while values closer to 0 indicate stronger similarity to historical graduate cases.

Since a regression model is not strictly constrained to remain within the target range, the predicted values are clipped to the interval from 0 to 1 to preserve interpretability as percentage-based risk scores.

In [21]:

```
# Predict continuous dropout-related risk scores
y_score = risk_model.predict(X_test)

# Keep scores within the interpretable 0-1 range
y_score = np.clip(y_score, 0, 1)
```

5.4. Create Initial Results Table

Before continuing with regression-oriented error analysis, an initial results table is created to store the observed outcomes, the predicted continuous risk scores, and a percentage-based version of the score.

This provides a consistent structure for the later residual, absolute error, and ranking analyses.

In [22]:

```
results_df = pd.DataFrame({
  "Actual Outcome": y_test.values,
  "Predicted Risk Score": y_score,
  "Risk Score (%)": y_score * 100
})

results_df["Actual Label"] = results_df["Actual Outcome"].map({
  1: "Dropout",
  0: "Graduate"
})

results_df.head(10)
```

Out[22]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label
0	0	0.010401	1.040075	Graduate
1	0	0.162577	16.257693	Graduate
2	1	0.340720	34.071973	Dropout
3	0	0.283259	28.325882	Graduate
4	0	0.600824	60.082411	Graduate
5	0	0.027494	2.749365	Graduate
6	0	0.221548	22.154784	Graduate
7	1	0.957530	95.753018	Dropout
8	0	0.501984	50.198402	Graduate
9	1	0.753585	75.358542	Dropout

5.5. Evaluate the Ethical Gradient Boosting

Before using the predicted risk scores in later analysis, the selected model is evaluated on the test set. Because the notebook is framed as a regression-based risk estimation task, the main evaluation focuses on regression-oriented metrics and score quality rather than only hard classification performance.

In [23]:

```
mae = mean_absolute_error(y_test, y_score)
mse = mean_squared_error(y_test, y_score)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_score)
brier = brier_score_loss(y_test, y_score)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R²:", r2)
print("Brier Score:", brier)
```

```
MAE: 0.18649891200557894
MSE: 0.08151133514441027
RMSE: 0.28550190042171397
R²: 0.65774434004704
Brier Score: 0.08151133514441027
```

5.6. Residual Analysis

In addition to summary metrics such as MAE, RMSE, and R^2, a regression-based risk model should also be evaluated by examining its residuals.

A residual is the difference between the observed outcome and the predicted risk score:

- positive residuals indicate that the model underestimated dropout-related risk
- negative residuals indicate that the model overestimated dropout-related risk

Although the observed target remains binary, residual analysis still provides a useful regression-oriented view of how the continuous score behaves across the test set.

Because the observed target takes only the values 0 and 1, the residual structure in this notebook differs from that of a traditional regression problem with a naturally continuous outcome.

In [24]:
```
residuals = y_test - y_score

residuals_df = pd.DataFrame({
  "Actual Outcome": y_test.values,
  "Predicted Risk Score": y_score,
  "Residual": residuals
})

residuals_df.head(10)
```

Out[24]:

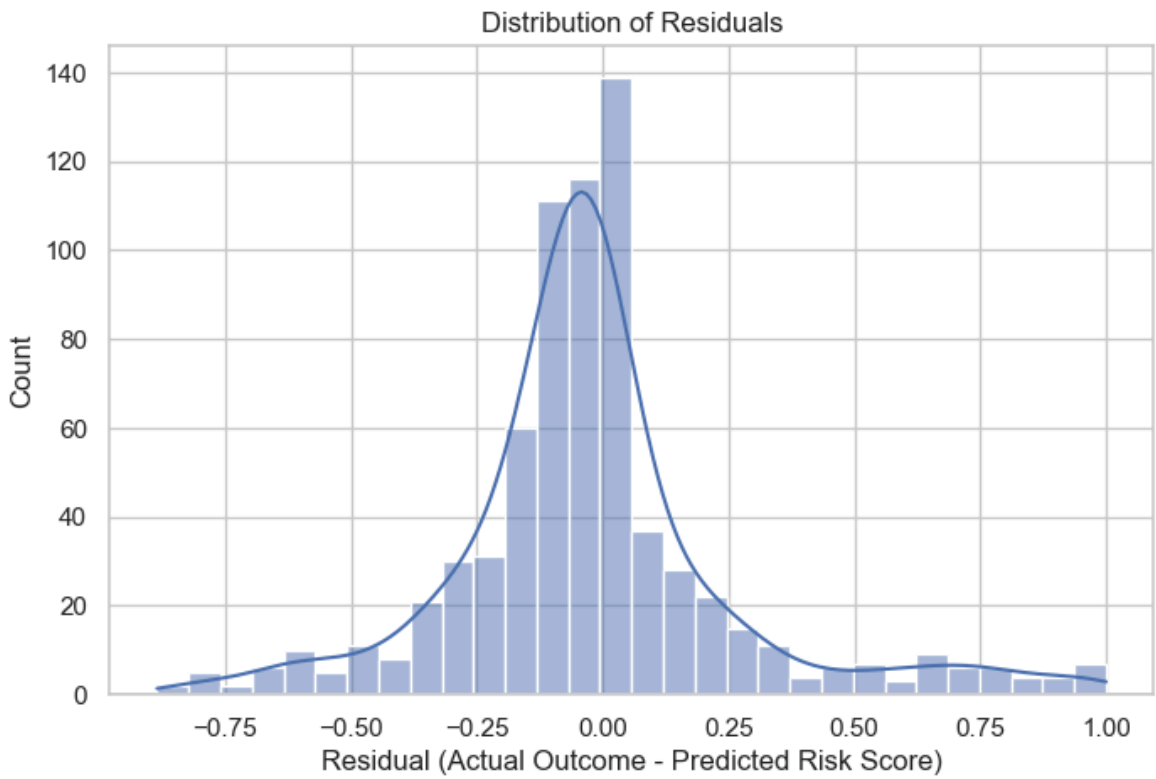
	Actual Outcome	Predicted Risk Score	Residual
3622	0	0.010401	-0.010401
4325	0	0.162577	-0.162577
1558	1	0.340720	0.659280
3458	0	0.283259	-0.283259
4291	0	0.600824	-0.600824
2224	0	0.027494	-0.027494
1451	0	0.221548	-0.221548
2914	1	0.957530	0.042470
3405	0	0.501984	-0.501984
605	1	0.753585	0.246415

5.6.1. Distribution of Residuals

The distribution of residuals helps show whether most prediction errors are small or whether large overestimation and underestimation occur frequently.

Residuals close to 0 indicate that the predicted risk score is relatively close to the observed outcome, while larger positive or negative values indicate stronger mismatch.

In [25]:
```
plt.figure(figsize=(8,5))
sns.histplot(residuals_df["Residual"], bins=30, kde=True)

plt.title("Distribution of Residuals")
plt.xlabel("Residual (Actual Outcome - Predicted Risk Score)")
plt.ylabel("Count")

plt.show()
```

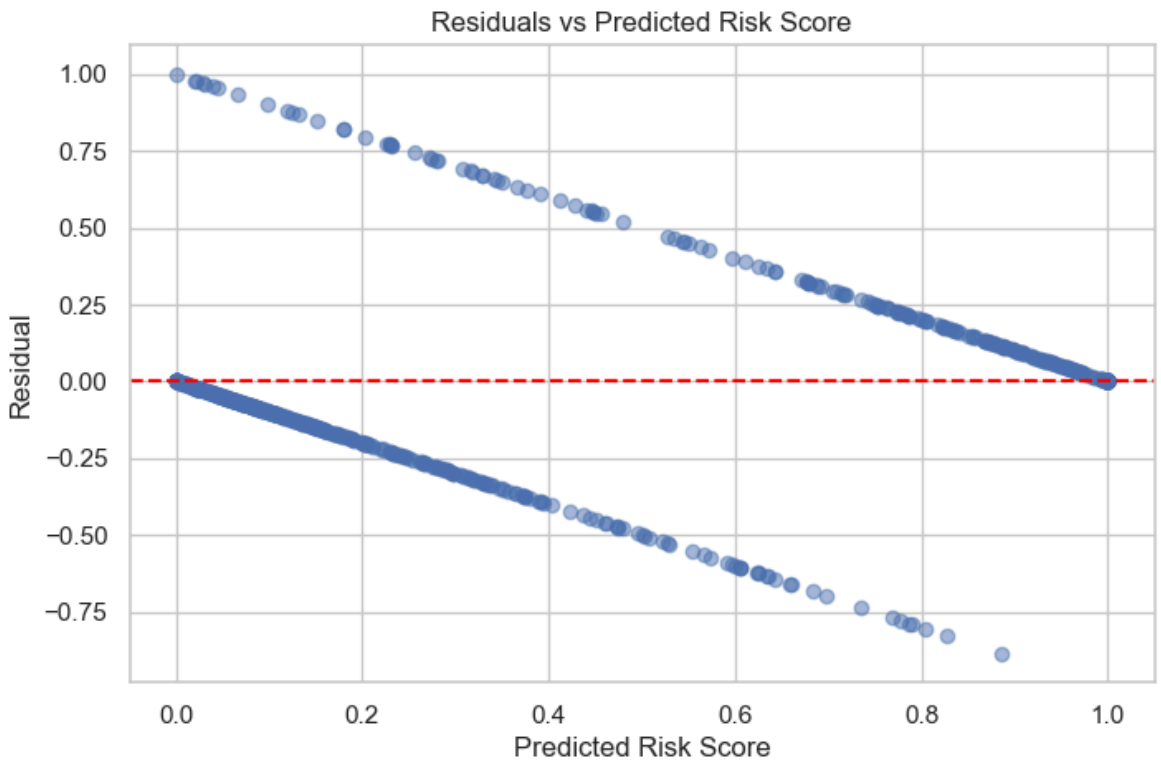


5.6.2. Residuals by Predicted Risk Score

A scatter plot of residuals against predicted risk score helps show whether the model behaves differently across the score range.

This is useful for checking how prediction error changes across the score range and for identifying where the model tends to underestimate dropout-related risk or overestimate it.

In [26]:
```
plt.figure(figsize=(8,5))
plt.scatter(residuals_df["Predicted Risk Score"], residuals_df["Residual"], alph

plt.axhline(0, color="red", linestyle="--")
plt.title("Residuals vs Predicted Risk Score")
plt.xlabel("Predicted Risk Score")
plt.ylabel("Residual")

plt.show()
```



In [27]:
```
residuals_df["Actual Label"] = residuals_df["Actual Outcome"].map({
  1: "Dropout",
  0: "Graduate"
})
```

5.6.3. Residual Summary by Actual Outcome

Residuals can also be summarized separately for students who actually dropped out and students who actually graduated.

This makes it easier to see whether the model tends to underpredict dropout cases or overpredict graduate cases more strongly in one group than the other.

In [28]:
```
residuals_df.groupby("Actual Label")["Residual"].describe()
```

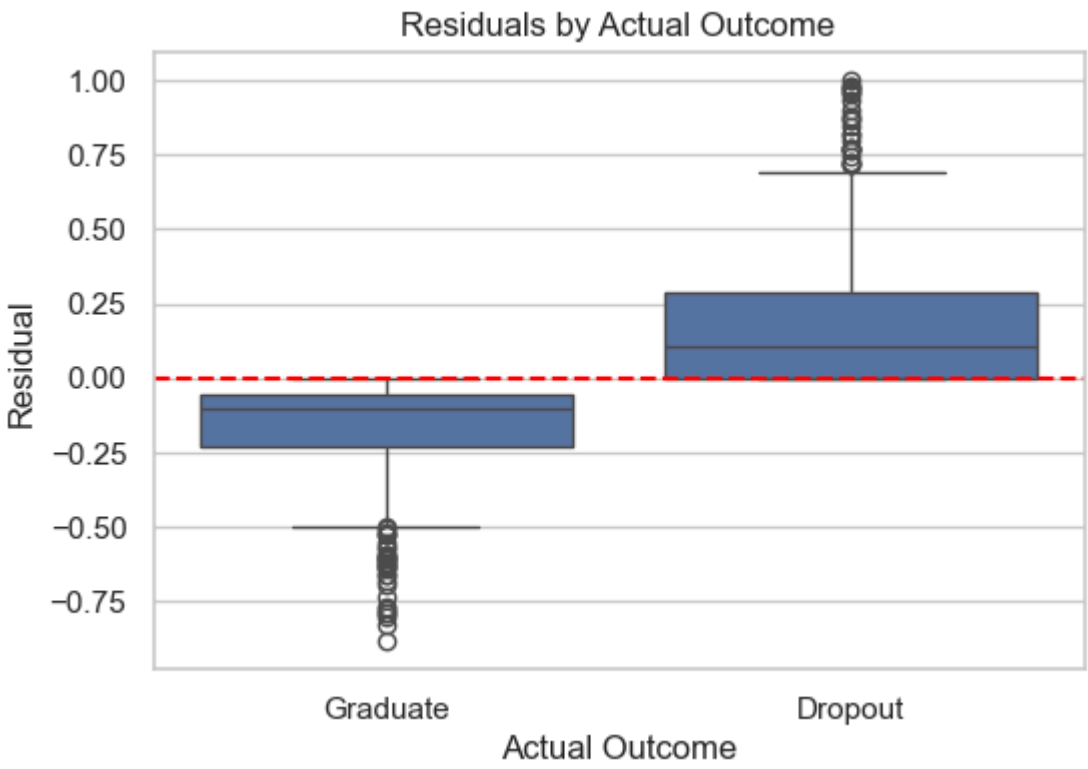
Out[28]:

Actual Label	count	mean	std	min	25%	50%	75%	max
Dropout	284.0	0.209792	0.266864	0.000000	0.000000	0.102968	0.284867	1.0
Graduate	442.0	-0.171532	0.175082	-0.885555	-0.233537	-0.105608	-0.054916	0.0

The mean residual is positive for actual dropout cases and negative for actual graduate cases, which reflects the model's tendency to assign lower-than-observed scores to some dropouts and higher-than-observed scores to some graduates.

In [29]:
```
plt.figure(figsize=(6,4))
sns.boxplot(data=residuals_df, x="Actual Label", y="Residual")

plt.axhline(0, color="red", linestyle="--")
```

```
plt.title("Residuals by Actual Outcome")
plt.xlabel("Actual Outcome")
plt.ylabel("Residual")

plt.show()
```



5.7. Absolute Error Analysis

While residuals show whether the model **underestimates** or **overestimates** dropout-related risk, absolute error focuses only on the size of the prediction error.

This is useful because positive and negative residuals can offset each other when viewed together, while absolute error directly shows how far each predicted risk score is from the observed outcome.

In [30]:
```
results_df["Absolute Error"] = np.abs(results_df["Actual Outcome"] - results_df[
results_df["Squared Error"] = (results_df["Actual Outcome"] - results_df["Predic

results_df.head(10)
```

Out[30]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label	Absolute Error	Squared Error
0	0	0.010401	1.040075	Graduate	0.010401	0.000108
1	0	0.162577	16.257693	Graduate	0.162577	0.026431
2	1	0.340720	34.071973	Dropout	0.659280	0.434650
3	0	0.283259	28.325882	Graduate	0.283259	0.080236
4	0	0.600824	60.082411	Graduate	0.600824	0.360990
5	0	0.027494	2.749365	Graduate	0.027494	0.000756
6	0	0.221548	22.154784	Graduate	0.221548	0.049083
7	1	0.957530	95.753018	Dropout	0.042470	0.001804
8	0	0.501984	50.198402	Graduate	0.501984	0.251988
9	1	0.753585	75.358542	Dropout	0.246415	0.060720

5.7.1. Error Summary by Actual Outcome

To better understand how strongly the prediction error differs between outcome groups, the absolute and squared errors are summarized separately for students who actually dropped out and students who actually graduated.

In [31]:
```
results_df.groupby("Actual Label")[["Absolute Error", "Squared Error"]].describe
```

Out[31]:

	Absolute Error								
	count	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**	**cour**
Actual Label									
Dropout	284.0	0.209792	0.266864	0.0	0.000000	0.102968	0.284867	1.000000	284.
Graduate	442.0	0.171532	0.175082	0.0	0.054916	0.105608	0.233537	0.885555	442.

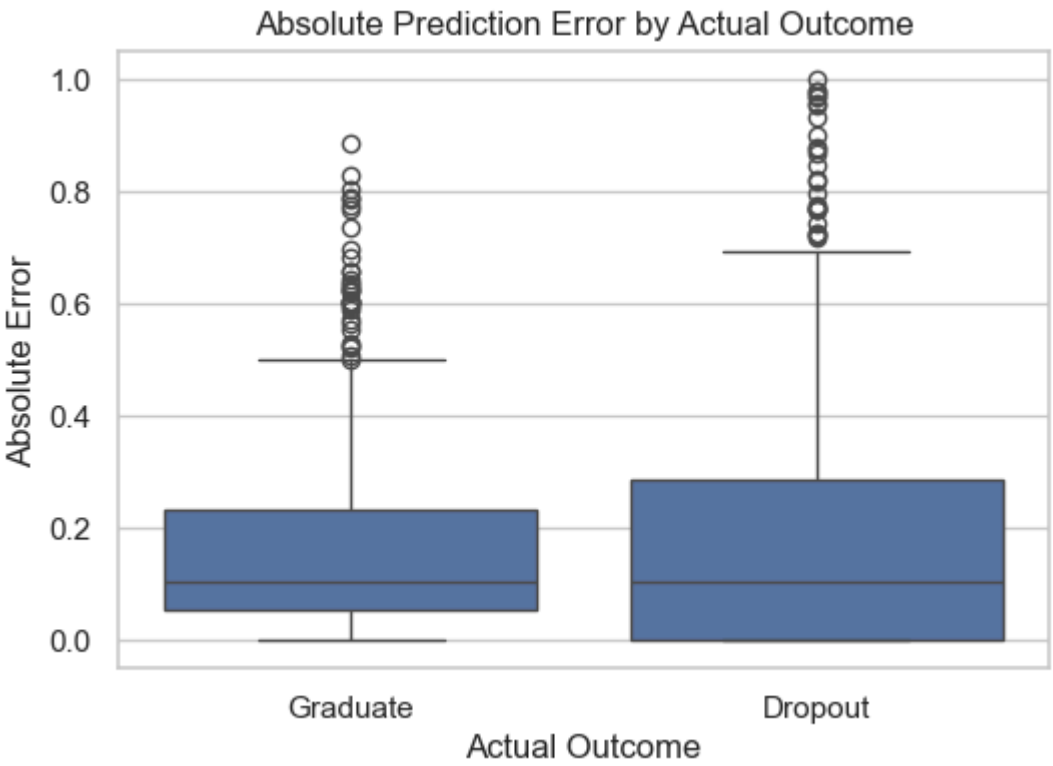
5.7.2. Absolute Error by Actual Outcome

A boxplot of absolute error helps compare how large the prediction errors are across the two actual outcome groups.

Smaller values indicate that the predicted risk score is closer to the observed outcome, while larger values indicate a stronger mismatch.

In [32]:
```
plt.figure(figsize=(6,4))
sns.boxplot(data=results_df, x="Actual Label", y="Absolute Error")

plt.title("Absolute Prediction Error by Actual Outcome")
plt.xlabel("Actual Outcome")
plt.ylabel("Absolute Error")
```

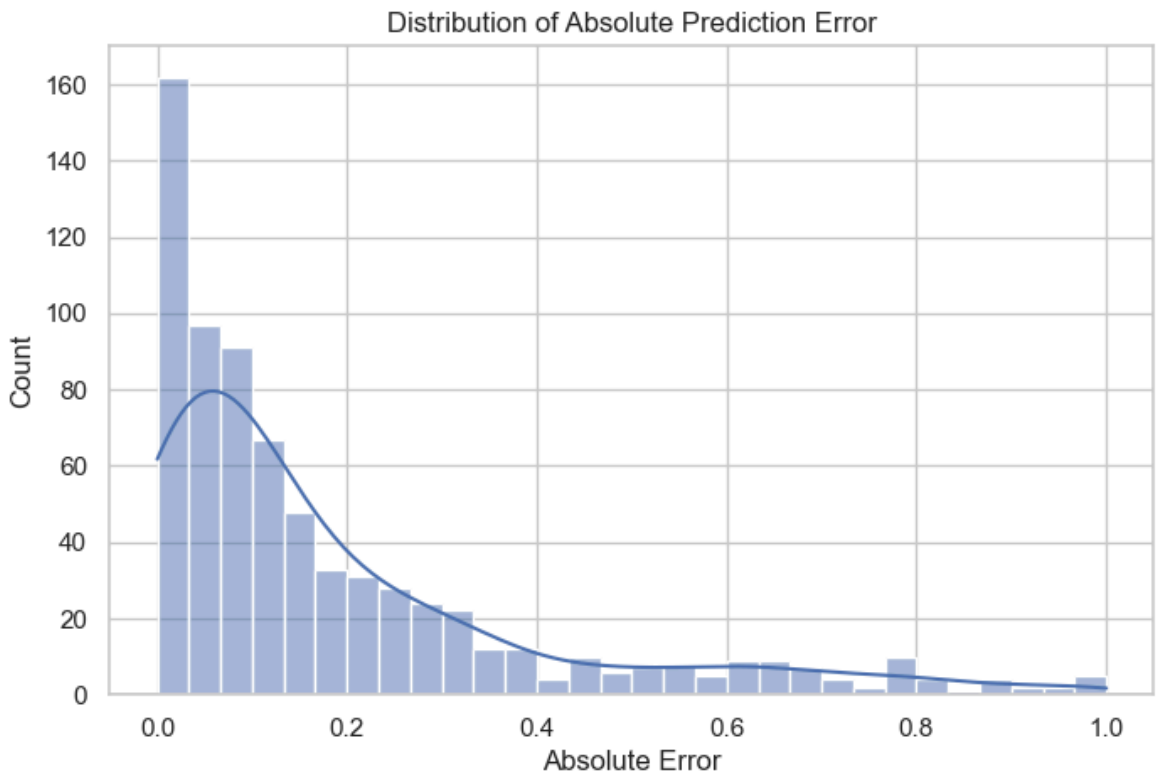
```
plt.show()
```



5.7.3. Distribution of Absolute Error

The overall distribution of absolute error shows whether most prediction errors remain relatively small or whether a substantial number of students receive scores that are far from the observed outcome.

In [33]:
```
plt.figure(figsize=(8,5))
sns.histplot(results_df["Absolute Error"], bins=30, kde=True)

plt.title("Distribution of Absolute Prediction Error")
plt.xlabel("Absolute Error")
plt.ylabel("Count")

plt.show()
```



5.8. Interpretation

The regression-based evaluation shows that the selected `Ethical Gradient Boosting Regressor` produces continuous dropout-related risk scores that remain meaningfully aligned with the observed student outcomes.

The residual analysis provides insight into the direction of prediction error. It shows that actual dropout cases tend to have positive residuals, while actual graduate cases tend to have negative residuals. This is expected because the observed target is binary, but it also highlights where the model underestimates or overestimates dropout-related risk.

The absolute error analysis complements this by focusing on the size of the prediction error rather than its direction. The results show that many students receive scores with relatively small error, while a smaller group of students shows substantially larger mismatch between predicted score and observed outcome. The error distribution is therefore concentrated near zero but right-skewed, which suggests that the model performs reasonably well for many cases while still making larger mistakes for some students.

The summary statistics also show that actual dropout cases have somewhat higher absolute and squared errors on average than actual graduate cases. This suggests that dropout cases may be slightly harder for the model to score accurately, even though the overall risk signal remains useful.

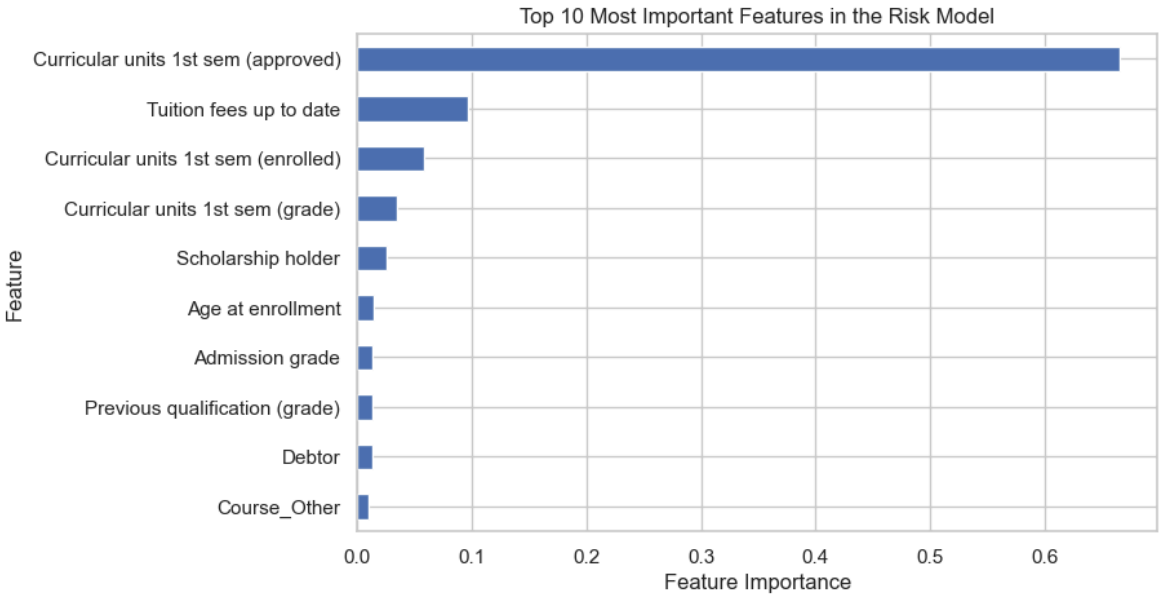
Overall, these results support the use of the model for continuous risk scoring, percentage-based interpretation, ranking, and later support-oriented analysis.

5.9. Feature Importance Analysis

In addition to evaluating score quality, it is also useful to examine which features contribute most strongly to the regression-based risk model.

`Feature importance` does not show causality, but it helps indicate which variables the model relies on most when generating dropout-related risk scores. This supports practical interpretability and helps assess whether the model is focusing on academically meaningful information.

In [34]:
```
feature_importance = pd.Series(risk_model.feature_importances_, index=X_train.co
top_features = feature_importance.sort_values(ascending=False).head(10)

plt.figure(figsize=(8,5))
top_features.sort_values().plot(kind="barh")

plt.title("Top 10 Most Important Features in the Risk Model")
plt.xlabel("Feature Importance")
plt.ylabel("Feature")

plt.show()
```



5.9.1. Interpretation

The feature importance analysis helps show which variables contribute most strongly to the regression-based risk score. Although these values should not be interpreted as causal effects, they provide a useful indication of the patterns the model relies on when identifying higher-risk and lower-risk student profiles.

This is especially relevant in a support-oriented context, because it helps assess whether the model is drawing primarily on meaningful academic and institutional variables rather than relying on sensitive background characteristics that were intentionally excluded in the ethical feature set.

6. Risk Score Construction

The main purpose of this notebook is to move beyond a simple binary outcome and translate the model's continuous regression output into a more interpretable dropout-related risk score.

In this notebook, each student receives a continuous score between 0 and 1. A higher score means that the student's profile is more similar to historical patterns associated with dropout, while a lower score indicates a profile more similar to students who graduate.

This does not mean that the model measures student `struggle` directly. Instead, the score should be understood as an estimated dropout-related risk level derived from the regression model.

6.1. Define the Risk Score

The predicted regression output is used as the student risk score.

This means that:

- a value close to **0** indicates **low** estimated dropout-related risk
- a value close to **1** indicates **high** estimated dropout-related risk

These continuous scores provide a more gradual interpretation of model output than a simple class label and are therefore more suitable for early warning and support-oriented decision-making.

6.2. Convert Risk Score to Percentage

To make the model output easier to interpret, the continuous risk scores are converted into **percentage form**.

For example:

- 0.20 becomes 20% estimated dropout-related risk
- 0.75 becomes 75% estimated dropout-related risk

In [35]:
```
risk_scores = y_score
risk_scores_percent = risk_scores * 100
```

6.3. Review the Results Table

To inspect the predicted scores more clearly, the results table created earlier is now revisited in the context of practical risk interpretation.

The table contains:

- the actual student outcome
- the predicted continuous risk score
- the percentage-based risk score

This makes the risk scoring framework more transparent and allows individual predictions to be reviewed more easily.

In [36]:
```
results_df.head(10)
```

Out[36]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label	Absolute Error	Squared Error
0	0	0.010401	1.040075	Graduate	0.010401	0.000108
1	0	0.162577	16.257693	Graduate	0.162577	0.026431
2	1	0.340720	34.071973	Dropout	0.659280	0.434650
3	0	0.283259	28.325882	Graduate	0.283259	0.080236
4	0	0.600824	60.082411	Graduate	0.600824	0.360990
5	0	0.027494	2.749365	Graduate	0.027494	0.000756
6	0	0.221548	22.154784	Graduate	0.221548	0.049083
7	1	0.957530	95.753018	Dropout	0.042470	0.001804
8	0	0.501984	50.198402	Graduate	0.501984	0.251988
9	1	0.753585	75.358542	Dropout	0.246415	0.060720

6.4. Review Human-Readable Outcome Labels

To make the results table easier to interpret, the binary observed outcomes are also shown as descriptive labels.

In [37]:
```
results_df.head(10)
```

Out[37]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label	Absolute Error	Squared Error
0	0	0.010401	1.040075	Graduate	0.010401	0.000108
1	0	0.162577	16.257693	Graduate	0.162577	0.026431
2	1	0.340720	34.071973	Dropout	0.659280	0.434650
3	0	0.283259	28.325882	Graduate	0.283259	0.080236
4	0	0.600824	60.082411	Graduate	0.600824	0.360990
5	0	0.027494	2.749365	Graduate	0.027494	0.000756
6	0	0.221548	22.154784	Graduate	0.221548	0.049083
7	1	0.957530	95.753018	Dropout	0.042470	0.001804
8	0	0.501984	50.198402	Graduate	0.501984	0.251988
9	1	0.753585	75.358542	Dropout	0.246415	0.060720

6.5. Sort Students by Risk Score

To better understand how the model ranks students, the results table can be sorted from highest to lowest predicted risk.

This makes it easier to inspect which students are being flagged as the most at risk.

In [38]:

```
results_df_sorted = results_df.sort_values(by="Risk Score (%)", ascending=False)

results_df_sorted.head(10)
```

Out[38]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label	Absolute Error	Squared Error
721	1	1.0	100.0	Dropout	0.0	0.0
324	1	1.0	100.0	Dropout	0.0	0.0
323	1	1.0	100.0	Dropout	0.0	0.0
328	1	1.0	100.0	Dropout	0.0	0.0
335	1	1.0	100.0	Dropout	0.0	0.0
41	1	1.0	100.0	Dropout	0.0	0.0
367	1	1.0	100.0	Dropout	0.0	0.0
39	1	1.0	100.0	Dropout	0.0	0.0
670	1	1.0	100.0	Dropout	0.0	0.0
308	1	1.0	100.0	Dropout	0.0	0.0

6.6. Interpretation

The results table shows that the selected model assigns each student a continuous dropout-related risk score rather than only a yes-or-no prediction.

This is useful because students can be ranked by estimated risk and compared more gradually. Some students receive very low scores close to 0, while others receive scores close to 1. Many students fall between these extremes, which makes the score more informative for early intervention than a hard category alone.

For this reason, the risk score provides a practical basis for ranking students and designing interpretable support-oriented risk categories.

7. Risk Score Distribution Analysis

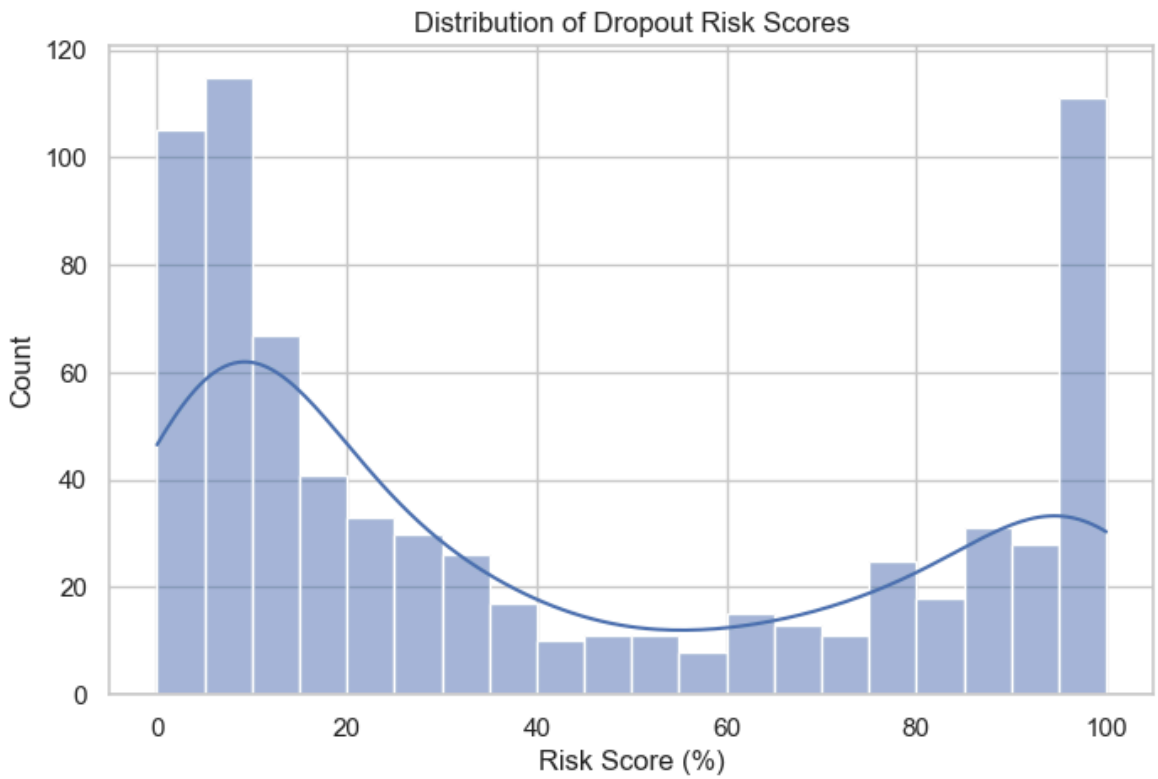
The risk score becomes more useful when its overall distribution is examined across the full test set. By analyzing how the predicted risk scores are distributed, it becomes possible to understand whether the model assigns clearly different risk levels to dropout and graduate cases.

This section explores the distribution of the generated risk scores, both overall and by actual student outcome. The goal is to evaluate whether the score behaves as a meaningful early warning indicator and whether dropout cases tend to receive higher estimated risk values than graduate cases.

7.1. Overall Distribution of Risk Scores

The first step is to inspect the overall distribution of the percentage-based risk scores across all students in the test set.

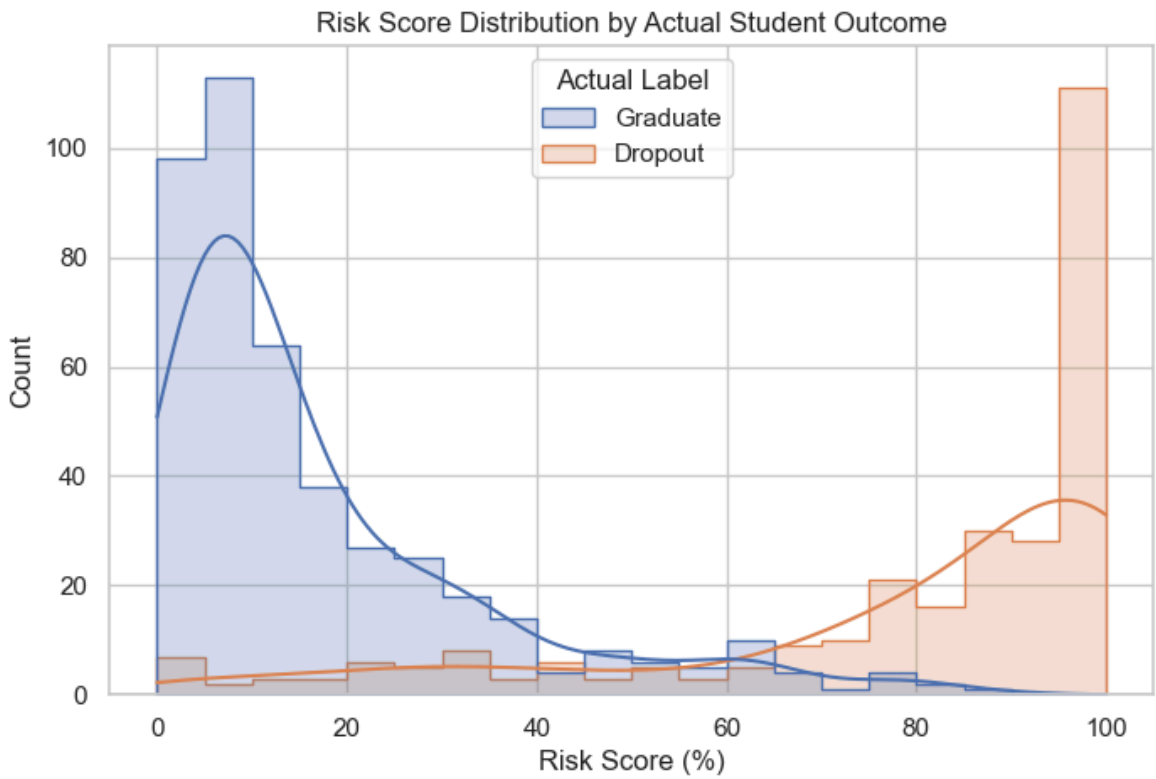
In [39]:
```
plt.figure(figsize=(8,5))
sns.histplot(results_df["Risk Score (%)"], bins=20, kde=True)

plt.title("Distribution of Dropout Risk Scores")
plt.xlabel("Risk Score (%)")
plt.ylabel("Count")

plt.show()
```



7.2. Risk Score Distribution by Actual Outcome

To better understand how well the risk score separates the two groups, the score distributions are compared for students who actually dropped out and students who actually graduated.

In [40]:
```
plt.figure(figsize=(8,5))
sns.histplot(
  data=results_df,
  x="Risk Score (%)",
  hue="Actual Label",
  bins=20,
  kde=True,
  element="step"
)

plt.title("Risk Score Distribution by Actual Student Outcome")
plt.xlabel("Risk Score (%)")
plt.ylabel("Count")

plt.show()
```



7.3. Summary Statistics by Actual Outcome

To complement the visual analysis, summary statistics are calculated for the risk scores in each actual outcome group.

In [41]:
```
results_df.groupby("Actual Label")["Risk Score (%)"].describe()
```

Out[41]:

Actual Label	count	mean	std	min	25%	50%	75%	m
Dropout	284.0	79.020768	26.686434	0.0	71.513260	89.703219	100.000000	100.0000
Graduate	442.0	17.153211	17.508155	0.0	5.491627	10.560802	23.353744	88.5554

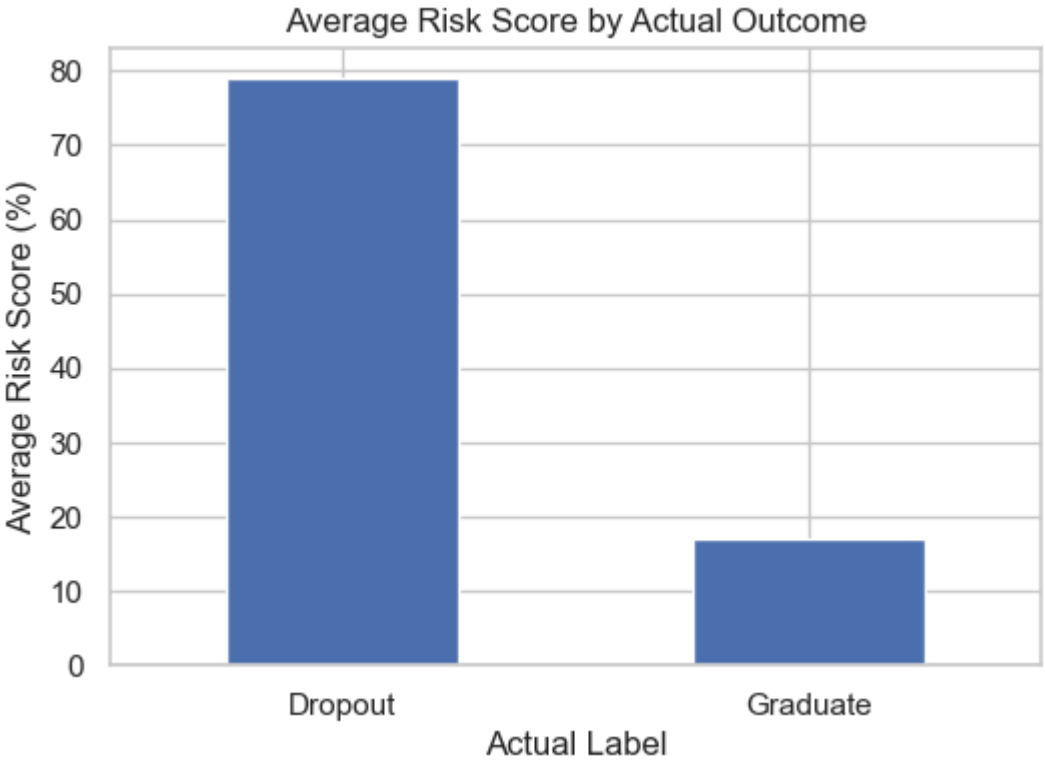
7.4. Average Risk Score by Actual Outcome

The mean risk score for each actual outcome group provides a simple summary of how differently the model scores graduates and dropouts.

In [42]:
```
avg_risk_by_outcome = results_df.groupby("Actual Label")["Risk Score (%)"].mean(

avg_risk_by_outcome
```

Out[42]:
```
Actual Label
Dropout     79.020768
Graduate    17.153211
Name: Risk Score (%), dtype: float64
```

In [43]:
```
plt.figure(figsize=(6,4))
avg_risk_by_outcome.plot(kind="bar")

plt.title("Average Risk Score by Actual Outcome")
plt.ylabel("Average Risk Score (%)")
plt.xticks(rotation=0)

plt.show()
```

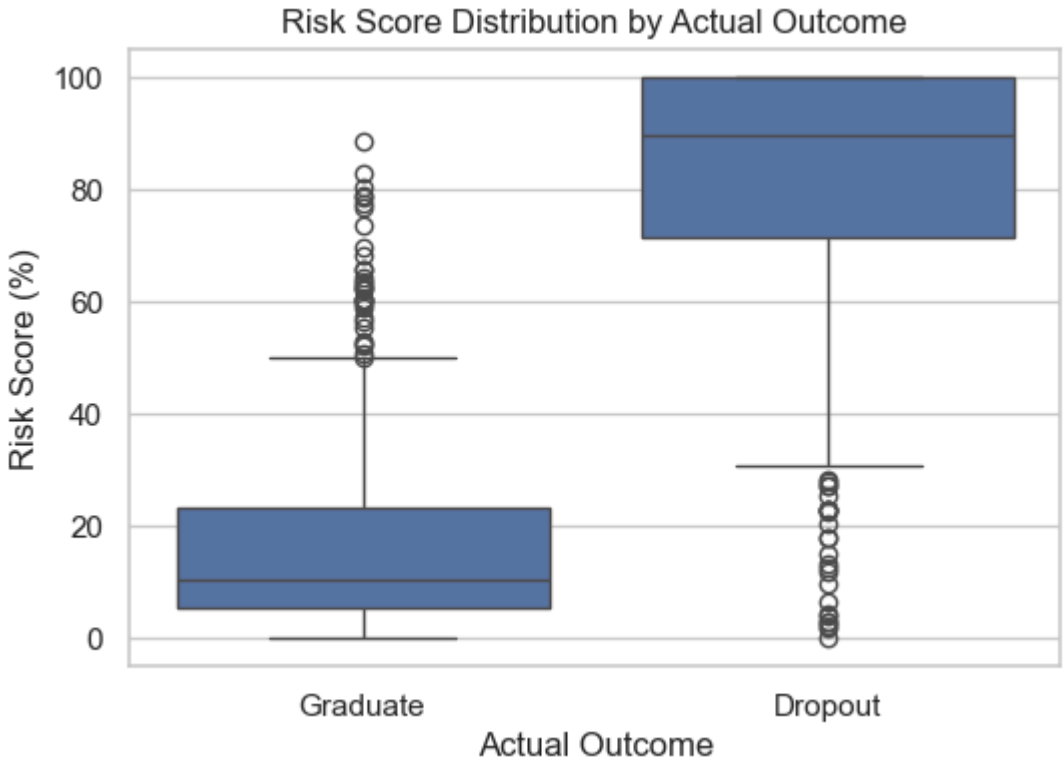


7.5. Boxplot of Risk Scores by Actual Outcome

In addition to the histogram-based comparison, a boxplot provides a clearer summary of how the continuous risk scores are distributed within each actual outcome group.

This makes it easier to compare the median risk score, the spread of the scores, and the amount of overlap between students who actually dropped out and students who graduated.

In [44]:
```
plt.figure(figsize=(6,4))
sns.boxplot(data=results_df, x="Actual Label", y="Risk Score (%)")

plt.title("Risk Score Distribution by Actual Outcome")
plt.xlabel("Actual Outcome")
plt.ylabel("Risk Score (%)")

plt.show()
```



7.6. Risk Score Quantiles

To further examine how the model spreads students across the continuous risk range, summary statistics with selected quantiles are calculated for the percentage-based risk score.

This helps show whether the predicted scores are concentrated in a narrow interval or meaningfully distributed across lower, middle, and higher estimated risk levels.

In [45]:
```
results_df["Risk Score (%)"].describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9])
```

Out[45]:
```
count    726.000000
mean      41.354845
std       37.111758
min        0.000000
10%        3.550647
25%        8.221748
50%       25.517125
75%       82.005870
90%      100.000000
max      100.000000
Name: Risk Score (%), dtype: float64
```

7.7. Interpretation

The distribution analysis shows that the predicted risk scores behave as a meaningful continuous dropout-related risk indicator. Students who actually dropped out generally receive substantially higher scores, while students who graduated are concentrated more strongly in the lower-risk range.

This pattern is visible in the histograms, the group summary statistics, the average score comparison, and the boxplot. Together, these results show that the model does not simply separate students into two extremes, but instead produces a score that varies meaningfully across the risk range.

The quantile summary further shows that the predicted scores are spread across the continuous scale rather than being concentrated in only a small interval. This is important because a useful regression-based risk score should support gradual interpretation, ranking, and comparison between students.

At the same time, some overlap remains between the two outcome groups. This suggests that the score should be interpreted as an early warning indicator rather than a certain prediction. Overall, the results support the use of the continuous score for ranking students, comparing relative risk levels, and supporting practical risk interpretation.

8. Risk Decile Analysis

A continuous risk score becomes more useful when it can meaningfully rank students from lower estimated risk to higher estimated risk.

One way to evaluate this is to divide the predicted risk scores into equally sized groups, or deciles, and then compare the observed dropout rate across these groups. If the regression-based risk score is behaving well, higher deciles should generally correspond to higher observed dropout frequencies.

This provides a more gradual and regression-oriented view of score quality than a single binary decision boundary.

Because many students receive identical or very similar predicted scores, especially near the lower and upper ends of the score range, duplicate cut points may occur. For this reason, the final grouping may contain fewer than ten distinct score bands.

In [46]:
```
results_df["Risk Decile"] = pd.qcut(
  results_df["Predicted Risk Score"],
  10,
  labels=False,
  duplicates="drop"
) + 1

results_df[["Predicted Risk Score", "Risk Score (%)", "Risk Decile"]].head(10)
```

Out[46]:

	Predicted Risk Score	Risk Score (%)	Risk Decile
0	0.010401	1.040075	1
1	0.162577	16.257693	5
2	0.340720	34.071973	6
3	0.283259	28.325882	6
4	0.600824	60.082411	7
5	0.027494	2.749365	1
6	0.221548	22.154784	5
7	0.957530	95.753018	9
8	0.501984	50.198402	7
9	0.753585	75.358542	8

8.1. Summary of Risk Deciles

For each score band, the following values are calculated:

- the average predicted risk score
- the observed dropout rate
- the number of students in the group

This makes it possible to compare how the model's continuous score relates to the actual observed outcome across the ranked score range.

In [47]:

```
decile_summary = results_df.groupby("Risk Decile").agg(
  Avg_Risk_Score=("Predicted Risk Score", "mean"),
  Observed_Dropout_Rate=("Actual Outcome", "mean"),
  Student_Count=("Actual Outcome", "count")
).reset_index()

decile_summary["Avg_Risk_Score"] = decile_summary["Avg_Risk_Score"] * 100
decile_summary["Observed_Dropout_Rate"] = decile_summary["Observed_Dropout_Rate"

decile_summary
```

Out[47]:

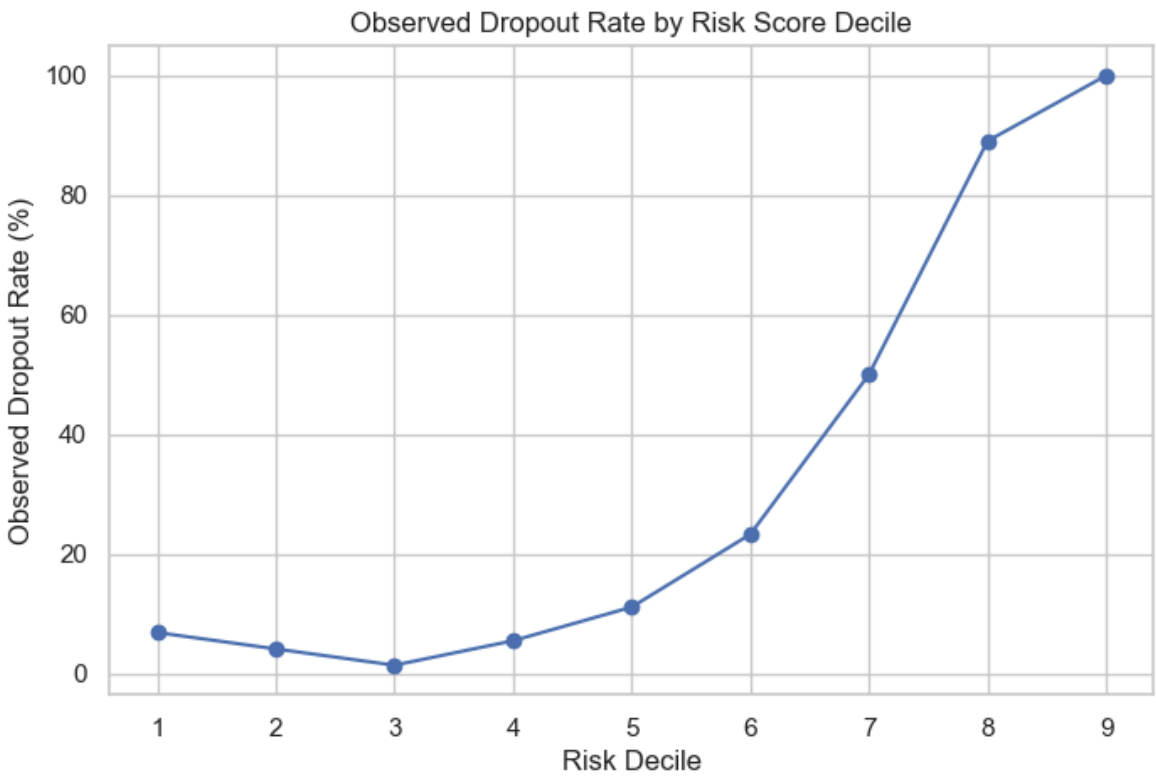
	Risk Decile	Avg_Risk_Score	Observed_Dropout_Rate	Student_Count
0	1	1.455026	6.849315	73
1	2	5.204098	4.109589	73
2	3	8.247593	1.388889	72
3	4	12.436218	5.479452	73
4	5	19.755785	11.111111	72
5	6	32.217797	23.287671	73
6	7	57.699631	50.000000	72
7	8	81.767497	89.041096	73
8	9	97.504235	100.000000	145

8.2. Observed Dropout Rate by Risk Decile

If the continuous risk score provides meaningful ranking information, the observed dropout rate should generally increase as the risk decile increases.

In [48]:

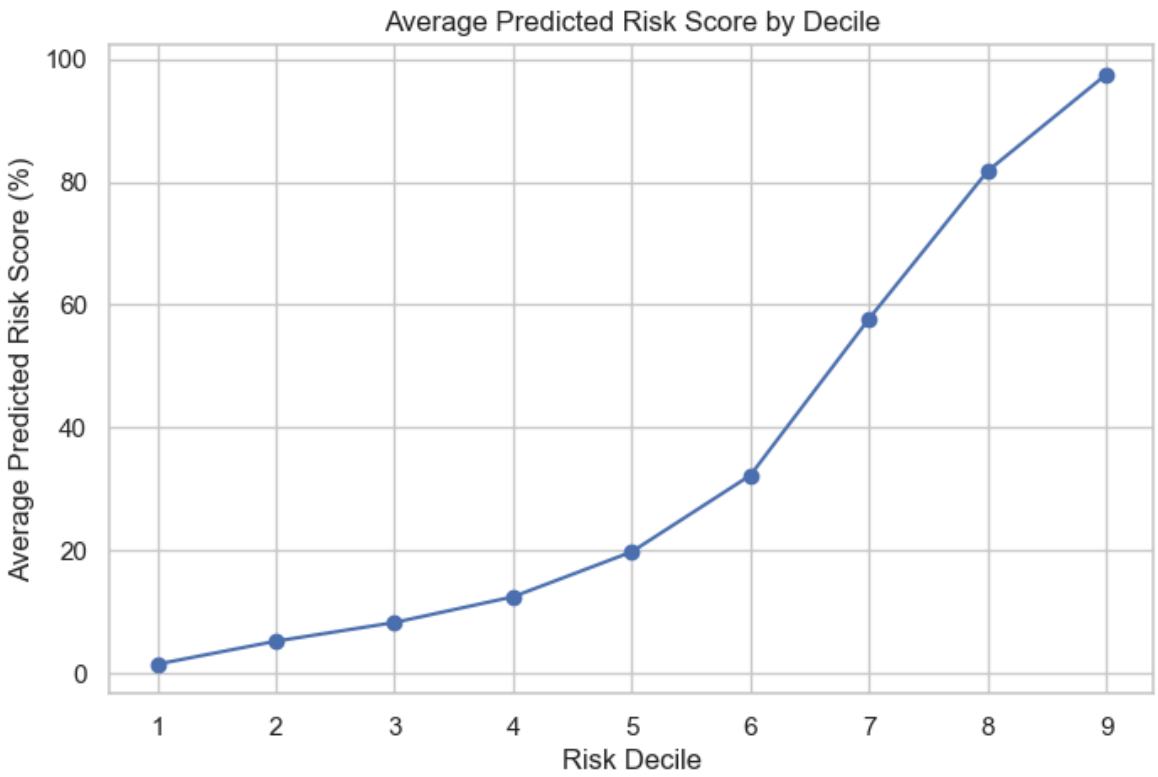
```
plt.figure(figsize=(8,5))
plt.plot(decile_summary["Risk Decile"], decile_summary["Observed_Dropout_Rate"],

plt.title("Observed Dropout Rate by Risk Score Decile")
plt.xlabel("Risk Decile")
plt.ylabel("Observed Dropout Rate (%)")

plt.show()
```



8.3. Average Predicted Risk Score by Decile

The average predicted risk score in each decile provides a direct view of how the model spreads students across the score range.

In [49]:
```
plt.figure(figsize=(8,5))
plt.plot(decile_summary["Risk Decile"], decile_summary["Avg_Risk_Score"], marker

plt.title("Average Predicted Risk Score by Decile")
plt.xlabel("Risk Decile")
plt.ylabel("Average Predicted Risk Score (%)")

plt.show()
```



In [50]:
```
decile_summary["Gap"] = decile_summary["Observed_Dropout_Rate"] - decile_summary
decile_summary
```

Out[50]:

	Risk Decile	Avg_Risk_Score	Observed_Dropout_Rate	Student_Count	Gap
0	1	1.455026	6.849315	73	5.394289
1	2	5.204098	4.109589	73	-1.094509
2	3	8.247593	1.388889	72	-6.858704
3	4	12.436218	5.479452	73	-6.956766
4	5	19.755785	11.111111	72	-8.644674
5	6	32.217797	23.287671	73	-8.930126
6	7	57.699631	50.000000	72	-7.699631
7	8	81.767497	89.041096	73	7.273599
8	9	97.504235	100.000000	145	2.495765

The gap between observed dropout rate and average predicted risk score is also calculated to show where the model tends to underestimate or overestimate group-level risk.

8.4. Interpretation

The risk decile analysis provides a more gradual evaluation of the regression-based risk score by examining whether students with higher predicted scores also show higher observed dropout rates.

Overall, the results show that the score behaves as a meaningful continuous ranking signal. The average predicted risk score increases steadily across the score bands, and the observed dropout rate also rises strongly, especially in the upper part of the ranking. This indicates that the model is not only separating students into broad groups, but also ordering them in a way that reflects increasing dropout-related risk.

The lower score bands show some small irregular variation in observed dropout rate, which is not unusual in relatively low-risk groups where the differences between students are smaller. In contrast, the upper score bands show a very clear increase in observed dropout frequency, rising from around 23% in the middle-upper range to 50%, then 89%, and finally 100% in the highest score band.

The comparison between average predicted risk and observed dropout rate also shows that the score is reasonably aligned with empirical outcomes, although some group-level underestimation and overestimation remains in specific bands. Overall, the decile analysis supports the use of the predicted score as a practical and ordered early warning indicator for continuous risk scoring.

9. Risk Category Design

Although the continuous risk score provides the most detailed view of student risk, it may still be useful in practice to translate the score into broader and easier-to-

communicate risk categories.

The previous decile analysis showed that the predicted score behaves as a meaningful ordered risk signal across the score range. Building on that result, this section introduces a simplified category framework that groups students into broader bands of low, medium, and high estimated dropout-related risk.

These categories are not intended to replace the underlying continuous score. Instead, they provide a more practical communication layer that can help support monitoring, review, and intervention planning.

9.1. Define Risk Categories

To make the continuous risk scores easier to communicate in practice, three broader risk bands are defined:

- Low Risk: 0% to below 30%
- Medium Risk: 30% to below 60%
- High Risk: 60% and above

These score intervals are used as an initial practical framework for grouping students based on their estimated dropout-related risk.

In [51]:
```
def assign_risk_category(score):
  if score < 30:
    return "Low Risk"
  elif score < 60:
    return "Medium Risk"
  else:
    return "High Risk"
```

In [52]:
```
results_df["Risk Category"] = results_df["Risk Score (%)"].apply(assign_risk_cat

results_df.head(10)
```

Out[52]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label	Absolute Error	Squared Error	Risk Decile	Risk Category
0	0	0.010401	1.040075	Graduate	0.010401	0.000108	1	Low Risk
1	0	0.162577	16.257693	Graduate	0.162577	0.026431	5	Low Risk
2	1	0.340720	34.071973	Dropout	0.659280	0.434650	6	Medium Risk
3	0	0.283259	28.325882	Graduate	0.283259	0.080236	6	Low Risk
4	0	0.600824	60.082411	Graduate	0.600824	0.360990	7	High Risk
5	0	0.027494	2.749365	Graduate	0.027494	0.000756	1	Low Risk
6	0	0.221548	22.154784	Graduate	0.221548	0.049083	5	Low Risk
7	1	0.957530	95.753018	Dropout	0.042470	0.001804	9	High Risk
8	0	0.501984	50.198402	Graduate	0.501984	0.251988	7	Medium Risk
9	1	0.753585	75.358542	Dropout	0.246415	0.060720	8	High Risk

9.1.1. Initial Justification of Risk Band Boundaries

The risk bands used in this notebook are intended as an initial practical framework for interpreting the continuous dropout-related risk scores. The cutoffs of 30% and 60% were selected to create three clearly distinguishable groups:

- students with relatively **low** estimated dropout-related risk
- students with **moderate** estimated risk who may benefit from closer review
- students with **high** estimated risk who may require earlier support

These category boundaries were not derived from a formal optimization procedure. Instead, they were chosen as interpretable score intervals that make the model output easier to communicate in an early intervention context.

The previous decile analysis already showed that the predicted scores behave as an ordered risk signal across the score range. The purpose of the present categories is therefore not to prove score quality again, but to simplify that continuous ranking into a more practical support-oriented structure.

Their usefulness is evaluated in the following subsections by examining how students are distributed across the categories and how the observed dropout rate differs between the `Low Risk`, `Medium Risk`, and `High Risk` groups.

These categories should therefore be understood as exploratory and support-oriented rather than as fixed institutional rules.

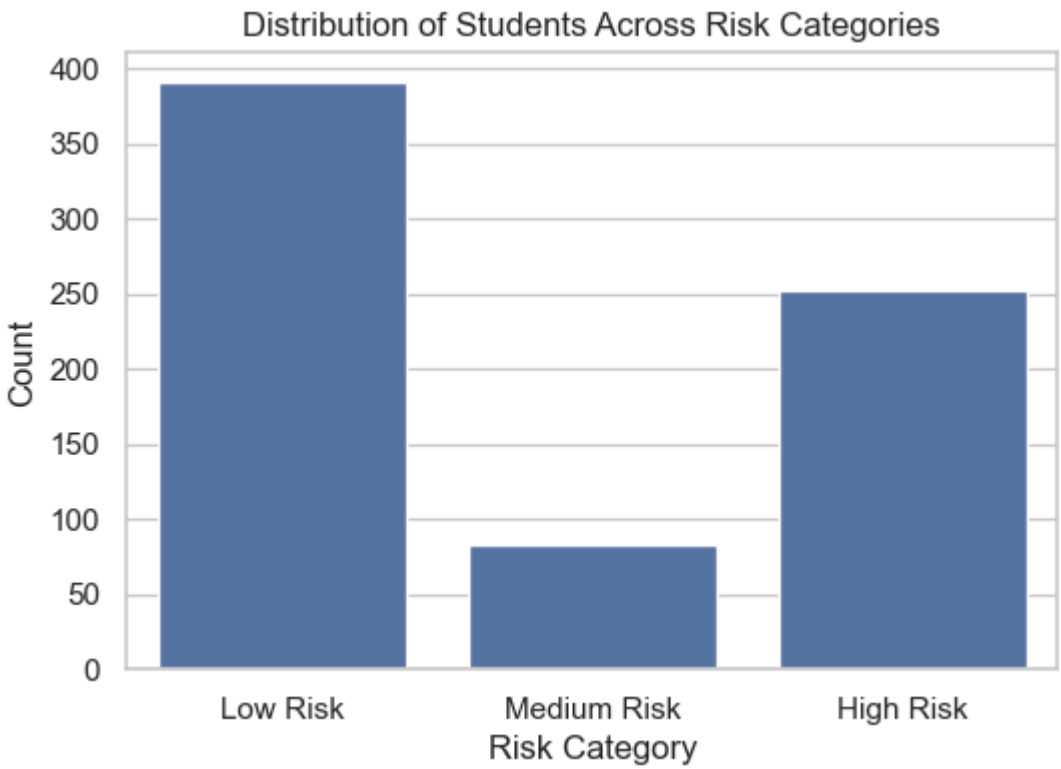
9.2. Distribution of Students Across Risk Categories

After defining the categories, the number of students in each risk group can be examined.

In [53]:
```
risk_category_counts = results_df["Risk Category"].value_counts()

risk_category_counts
```

Out[53]:
```
Risk Category
Low Risk       391
High Risk      252
Medium Risk     83
Name: count, dtype: int64
```

In [54]:
```
plt.figure(figsize=(6,4))
sns.countplot(x="Risk Category", data=results_df, order=["Low Risk", "Medium Ris

plt.title("Distribution of Students Across Risk Categories")
plt.xlabel("Risk Category")
plt.ylabel("Count")

plt.show()
```



9.3. Percentage of Students in Each Risk Category

To better understand the relative size of each group, the **percentage** of students in each category is also calculated.

In [55]:
```
risk_category_percentages = results_df["Risk Category"].value_counts(normalize=T
risk_category_percentages = risk_category_percentages.reindex(["Low Risk", "Medi

risk_category_percentages
```

Out[55]:
```
Risk Category
Low Risk       53.856749
Medium Risk    11.432507
High Risk      34.710744
Name: proportion, dtype: float64
```

9.4. Actual Dropout Rate Within Each Risk Category

A useful way to evaluate whether the simplified categories remain meaningful is to examine the actual dropout rate within each group.

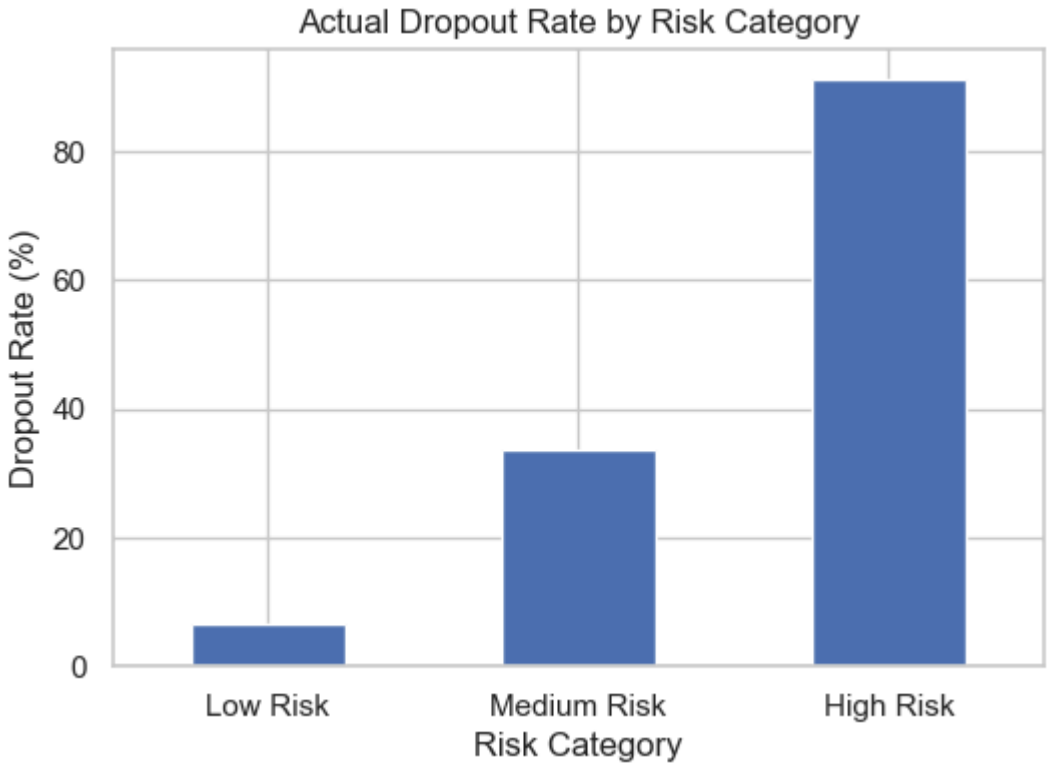
If the categories work well, the dropout rate should be:

- lowest in the low-risk group
- higher in the medium-risk group
- highest in the high-risk group

In [56]:
```
dropout_rate_by_category = results_df.groupby("Risk Category")["Actual Outcome"]

dropout_rate_by_category = dropout_rate_by_category.reindex(["Low Risk", "Medium
dropout_rate_by_category
```

Out[56]:
```
Risk Category
Low Risk        6.649616
Medium Risk    33.734940
High Risk      91.269841
Name: Actual Outcome, dtype: float64
```

In [57]:
```
plt.figure(figsize=(6,4))
dropout_rate_by_category.plot(kind="bar")

plt.title("Actual Dropout Rate by Risk Category")
plt.xlabel("Risk Category")
plt.ylabel("Dropout Rate (%)")
plt.xticks(rotation=0)

plt.show()
```



9.5. Crosstab of Risk Category and Actual Outcome

To see the relationship more directly, a crosstab can also be created.

In [58]:
```
pd.crosstab(results_df["Risk Category"], results_df["Actual Label"])
```

Out[58]:

Actual Label	Dropout	Graduate
Risk Category		
High Risk	230	22
Low Risk	26	365
Medium Risk	28	55

9.6. Interpretation

The risk category analysis shows that the continuous dropout-related risk score can be translated into broader and more practical student groups without losing its general meaning. The three categories correspond to clearly different levels of observed dropout risk.

The `Low Risk` group has a relatively low observed dropout rate, the `Medium Risk` group shows a noticeably higher rate, and the `High Risk` group is strongly associated with historical dropout outcomes. This indicates that the simplified category framework still preserves the main structure already visible in the continuous score and in the earlier decile analysis.

This is important because the continuous score remains the most detailed representation of estimated risk, while the categories provide a more accessible communication layer for practical use. Overall, the selected risk bands appear useful for summarizing the score into broader support-oriented groups for monitoring, review, and early intervention planning.

10. Calibration Analysis

The previous sections showed that the selected model produces meaningful continuous risk scores and clear separation between higher-risk and lower-risk students. However, a useful risk scoring system should not only rank students well, but should also produce scores that are reasonably aligned with observed dropout frequencies.

`Calibration analysis` is used here to examine whether higher predicted risk scores correspond to higher observed dropout rates. In other words, it helps assess whether the continuous score behaves like a reliable dropout-related risk indicator across different score ranges.

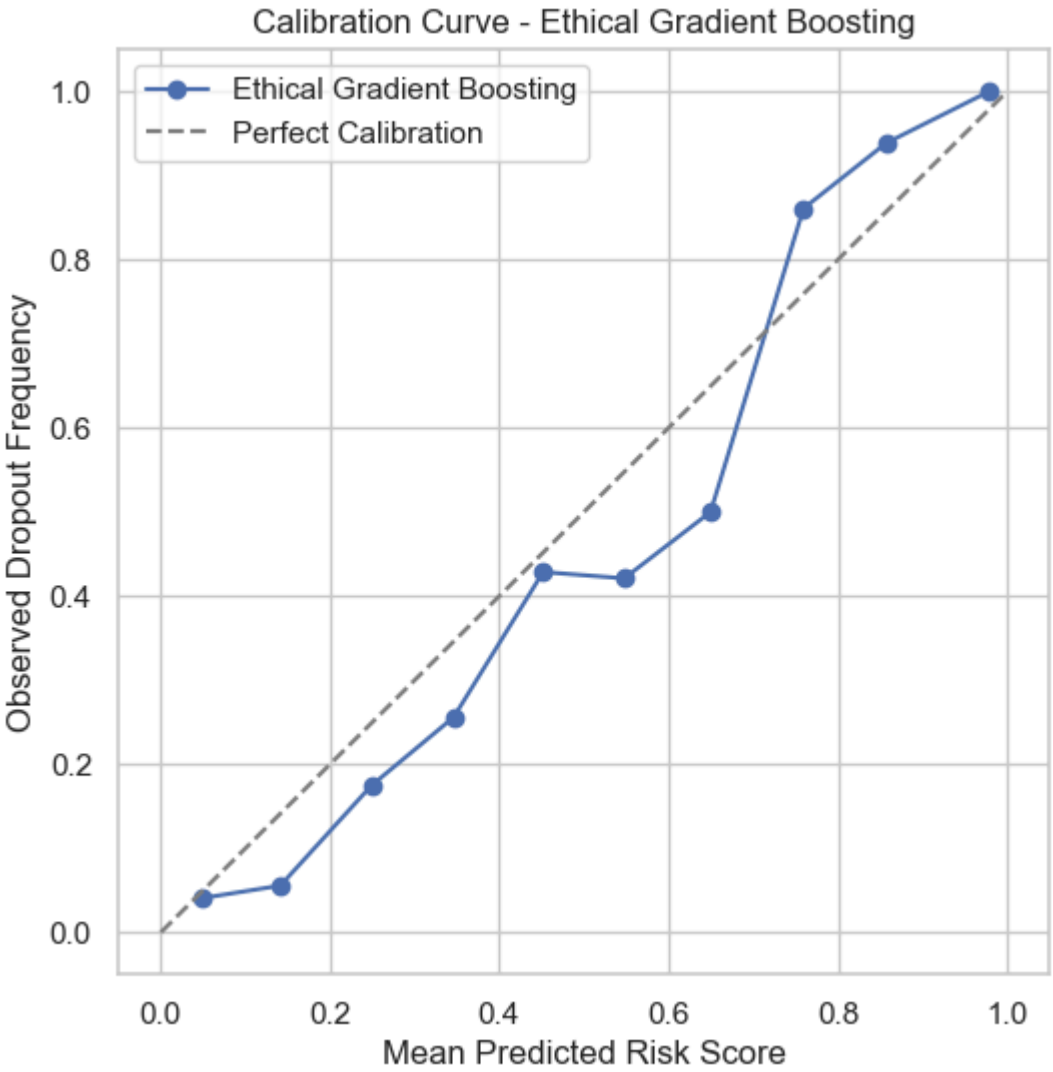
Because this notebook uses a regression model trained on a binary target, the calibration results should be interpreted cautiously. The scores are not guaranteed to be true probabilities, but calibration analysis can still help evaluate whether they behave similarly to meaningful empirical risk estimates.

10.1. Calibration Curve

A calibration curve compares the model's predicted risk scores with the actual observed frequency of dropout across different score ranges.

If the score behaves like a well-aligned empirical risk estimate, the calibration curve should stay reasonably close to the `diagonal reference line`.

In [59]:
```
prob_true, prob_pred = calibration_curve(y_test, y_score, n_bins=10)

plt.figure(figsize=(6,6))
plt.plot(prob_pred, prob_true, marker="o", label="Ethical Gradient Boosting")
plt.plot([0,1], [0,1], linestyle="--", color="gray", label="Perfect Calibration"

plt.xlabel("Mean Predicted Risk Score")
plt.ylabel("Observed Dropout Frequency")
plt.title("Calibration Curve - Ethical Gradient Boosting")
plt.legend()

plt.show()
```



10.2. Brier Score

`The Brier score` provides a numerical summary of how close the predicted risk scores are to the actual binary outcomes.

A lower Brier score indicates better alignment between the predicted risk scores and the observed binary outcomes, with 0 representing a perfect score.

In [60]:
```
brier = brier_score_loss(y_test, y_score)

print("Brier Score:", brier)
```

```
Brier Score: 0.08151133514441027
```

10.3. Interpretation

The calibration analysis suggests that the predicted risk scores are reasonably aligned with observed dropout frequencies across the test set. The calibration curve stays fairly close to the diagonal line, especially in some score ranges, which supports the practical usefulness of the score as a dropout-related risk indicator.

Some deviation is still visible, especially in the middle range, so the scores should not be interpreted as exact probabilities. Instead, they are better understood as approximate and practically useful risk estimates derived from the regression model.

The Brier score also supports the overall quality of the score alignment. Overall, the model appears suitable for support-oriented risk scoring, provided that the resulting percentages are interpreted carefully.

11. Support Capacity Simulation

In practice, universities may not have enough resources to provide intensive support to every student with elevated risk. For this reason, a risk scoring system can also be used to prioritize students when support capacity is limited.

This section simulates a simple support-allocation scenario by ranking students according to their predicted dropout-related risk score and then examining how many actual dropout cases are captured within the highest-risk groups.

11.1. Rank Students by Risk Score

The students in the test set are ranked from highest to lowest predicted risk.

In [65]:
```
results_df_ranked = results_df[[
  "Actual Outcome",
  "Predicted Risk Score",
  "Risk Score (%)",
  "Actual Label",
  "Risk Category"
]].sort_values(by="Risk Score (%)", ascending=False).reset_index(drop=True)

results_df_ranked.head(10)
```

Out[65]:

	Actual Outcome	Predicted Risk Score	Risk Score (%)	Actual Label	Risk Category
0	1	1.0	100.0	Dropout	High Risk
1	1	1.0	100.0	Dropout	High Risk
2	1	1.0	100.0	Dropout	High Risk
3	1	1.0	100.0	Dropout	High Risk
4	1	1.0	100.0	Dropout	High Risk
5	1	1.0	100.0	Dropout	High Risk
6	1	1.0	100.0	Dropout	High Risk
7	1	1.0	100.0	Dropout	High Risk
8	1	1.0	100.0	Dropout	High Risk
9	1	1.0	100.0	Dropout	High Risk

11.2. Simulate Support Capacity Levels

To explore how the model could be used in practice, several support-capacity levels are simulated:

- top 10% highest-risk students
- top 20% highest-risk students
- top 30% highest-risk students

For each level, the number of selected students and the number of actual dropout cases captured are calculated.

In [62]:
```
capacity_levels = [0.10, 0.20, 0.30]
capacity_results = []

total_dropouts = (results_df_ranked["Actual Outcome"] == 1).sum()
total_students = len(results_df_ranked)

for level in capacity_levels:
  n_students = int(total_students * level)
  selected = results_df_ranked.head(n_students)

  dropouts_captured = (selected["Actual Outcome"] == 1).sum()
  capture_rate = dropouts_captured / total_dropouts * 100

  capacity_results.append({
    "Support Capacity": f"Top {int(level * 100)}%",
    "Students Selected": n_students,
    "Actual Dropouts Captured": dropouts_captured,
    "Dropout Capture Rate (%)": capture_rate
  })

capacity_df = pd.DataFrame(capacity_results)
capacity_df
```

Out[62]:

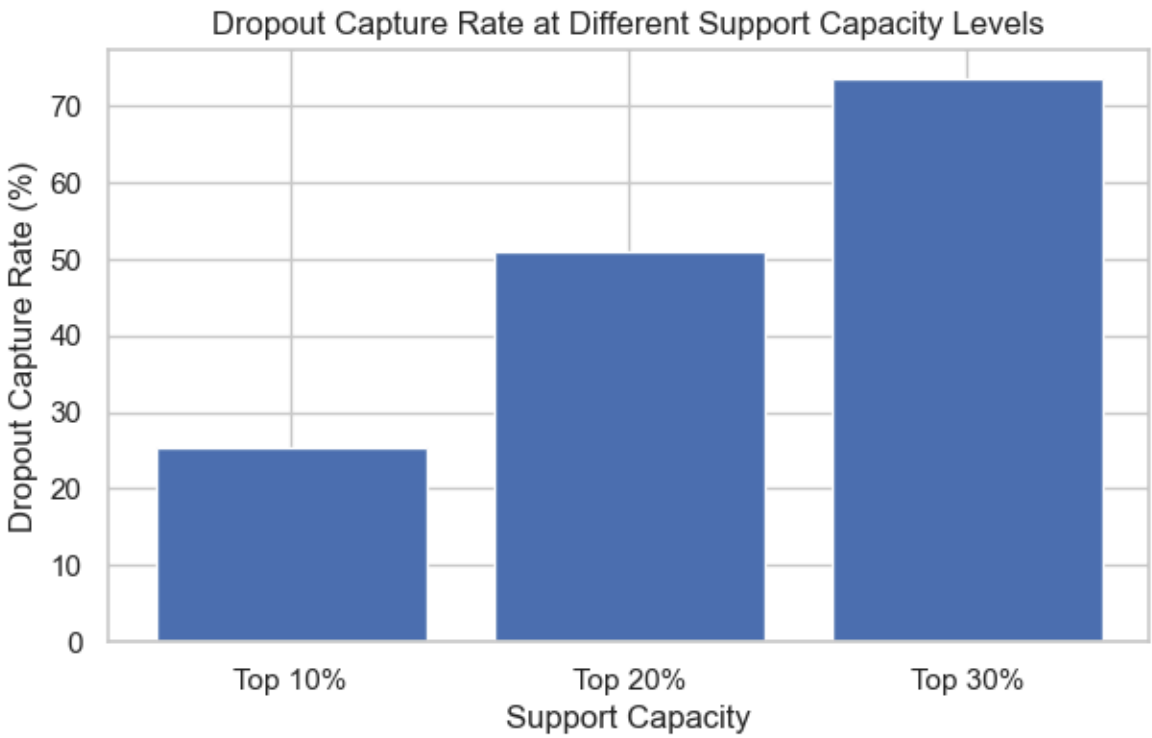
	Support Capacity	Students Selected	Actual Dropouts Captured	Dropout Capture Rate (%)
0	Top 10%	72	72	25.352113
1	Top 20%	145	145	51.056338
2	Top 30%	217	209	73.591549

11.3. Visualize Dropout Capture Rate

The capture rate shows what proportion of all actual dropout cases are included in the highest-risk student group at each support capacity level.

In [63]:
```
plt.figure(figsize=(7,4))
plt.bar(capacity_df["Support Capacity"], capacity_df["Dropout Capture Rate (%)"]

plt.title("Dropout Capture Rate at Different Support Capacity Levels")
```

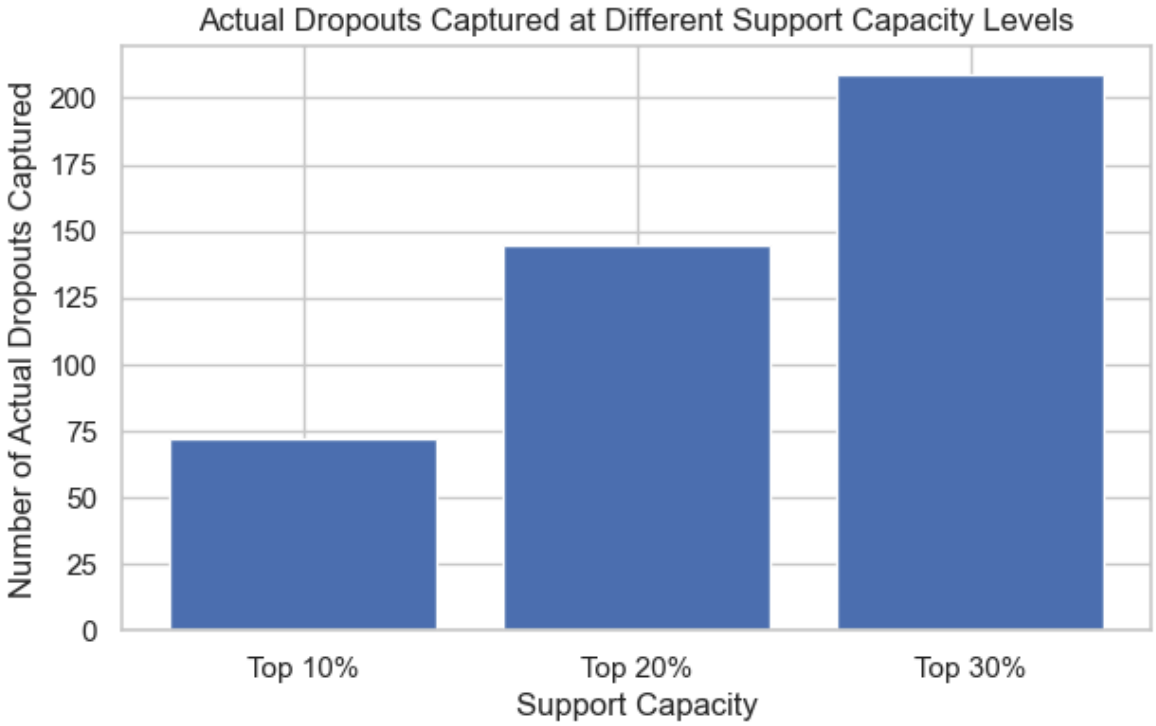
```
plt.xlabel("Support Capacity")
plt.ylabel("Dropout Capture Rate (%)")

plt.show()
```



11.4. Visualize Number of Dropouts Captured

This second chart shows the absolute number of dropout cases captured at each capacity level.

In [64]:
```
plt.figure(figsize=(7,4))
plt.bar(capacity_df["Support Capacity"], capacity_df["Actual Dropouts Captured"]

plt.title("Actual Dropouts Captured at Different Support Capacity Levels")
plt.xlabel("Support Capacity")
plt.ylabel("Number of Actual Dropouts Captured")

plt.show()
```



11.5. Interpretation

The support capacity simulation shows that the risk scores can be used effectively to prioritize students when support resources are limited. By focusing on the highest-risk students first, the model captures a substantial proportion of actual dropout cases.

If support can only be offered to the top 10% highest-risk students, the system captures 72 actual dropout cases, which represents approximately 25.4% of all dropout cases in the test set. Expanding support to the top 20% captures 145 dropout cases, or about 51.1% of the total. At the top 30% level, the system captures 209 dropout cases, corresponding to approximately 73.6% of all actual dropout cases.

These results suggest that the risk scoring framework can support practical intervention planning by helping institutions direct limited resources toward the students most strongly associated with dropout risk.

12. Practical and Ethical Interpretation

The previous sections showed that the selected model can generate meaningful dropout-related risk scores, group students into practical risk categories, support category-based interpretation, and help prioritize students when support capacity is limited.

This section reflects on how the risk scoring framework could be used in practice and what ethical precautions should be considered when interpreting the results.

12.1. Practical Use of Risk Scores

The risk scores developed in this notebook could support universities as an early warning tool for identifying students who may benefit from additional attention.

For example, institutions could use the scores to:

- monitor students in the `Low Risk` group through normal academic processes
- review students in the `Medium Risk` group more closely
- prioritize students in the `High Risk` group for earlier support or outreach

The support capacity simulation also shows that the scores can help allocate limited resources more effectively by focusing attention on the **highest-risk** students first.

At the same time, the model should not be understood as a replacement for academic advisors, teachers, or student support staff. Instead, it should be used as a decision-support tool that helps guide attention toward students who may need further review.

12.1.1. Example Support Actions by Risk Category

To illustrate how the risk categories could be used in practice, the table below presents example support responses that an institution might consider. These are only illustrative examples and should not be interpreted as fixed rules.

Risk Category	Example Interpretation	Possible Support Response
Low Risk	Student currently shows relatively low estimated dropout-related risk	Normal monitoring through standard academic processes
Medium Risk	Student shows moderate estimated risk and may benefit from additional review	Advisor check-in, review of academic progress, optional support contact
High Risk	Student shows high estimated risk and may require earlier attention	Priority outreach, academic guidance, financial support review, mentoring referral

The purpose of this table is not to automate interventions, but to show how score-based risk categories could help organize support efforts in a structured and responsible way.

12.2. Ethical Use of Predictions

Although the model produces useful risk estimates, these predictions should be interpreted carefully and used responsibly.

A **high risk** score does not mean that a student will definitely drop out. It only means that the student's profile is more strongly associated with historical patterns linked to dropout in the dataset. Similarly, a low risk score does not guarantee that a student is completely safe.

For this reason, the scores should be used to support helpful intervention, not punishment or exclusion. Their purpose should be to encourage academic guidance, mentoring, financial support, or other forms of assistance.

This notebook also uses the ethical feature set, which excludes sensitive demographic and family-background variables such as gender, nationality, and parental education or occupation. This reduces the likelihood that the model relies directly on potentially sensitive attributes when generating support-oriented predictions.

12.3. Limitations of Practical Use

Several limitations should still be considered when interpreting the results.

First, the dataset does not directly measure student struggle. The risk scores are derived from a regression model trained on a binary dropout outcome and should therefore be understood as dropout-related risk indicators, not as direct measures of personal difficulty.

Second, the model is based on historical data from a specific educational context. This means the results may not fully generalize to other institutions, student populations, or support systems.

Third, although the calibration analysis suggests that the predicted risk scores are reasonably aligned with observed dropout frequencies, the percentages should still be interpreted as approximate indicators rather than exact probabilities.

In addition, the results presented in this notebook are based on a single train/test split of the dataset. Although stratification was used and the selected model showed strong performance in this analysis, the exact category distributions, calibration pattern, and support-capacity outcomes may vary somewhat under different data splits.

Overall, the system appears useful as a support-oriented early warning framework, but it should always be applied with human judgment and institutional care.

12.4. Interpretation

The practical and ethical analysis suggests that the proposed risk scoring framework could be valuable as an early support tool in higher education. The model outputs are interpretable enough to guide monitoring, category-based review, and support prioritization, while the ethical feature selection helps reduce the risk of direct reliance on sensitive background variables.

At the same time, the results should not be treated as certain judgments about a student's future. The scores are best understood as structured warning signals that can help institutions provide timely and responsible support.

13. Conclusion

This notebook extended the previous student dropout prediction analysis by reframing a binary dropout outcome into a more interpretable continuous risk scoring framework for

early intervention.

Using the selected `Ethical Gradient Boosting Regressor`, predicted risk scores were converted into `percentage-based dropout-related risk scores` and examined through multiple practical perspectives. The analysis showed that these scores meaningfully separate dropout and graduate cases, support the creation of clear risk categories, and provide useful guidance for support-oriented interpretation and intervention prioritization.

The results also showed that the risk categories correspond to clearly different levels of observed dropout risk, while the support capacity simulation demonstrated that the highest-risk students contain a large share of actual dropout cases. In addition, the calibration analysis suggested that the predicted risk scores are reasonably aligned with observed dropout frequencies and can serve as practical risk indicators.

Overall, this notebook shows that a binary dropout outcome can be reframed into a continuous risk scoring workflow by training a regression model on the binary target and interpreting its bounded output as dropout-related risk. Rather than relying only on a final yes-or-no prediction, the model can support universities with a more gradual view of student risk that may help guide earlier and more targeted support.

At the same time, the results should still be interpreted carefully. The scores are not direct measurements of student struggle, and they should not be used as final judgments. Instead, they are best understood as dropout-related support indicators that can assist human decision-making in a more responsible and practical way.